

FREELANCEHUB

Online Freelance Job Management System

Project Report

Submitted by

Varsha Shaji

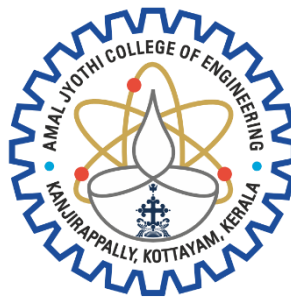
Reg. No.: AJC20MCA-I056

In Partial fulfillment for the Award of the Degree of

INTEGRATED MASTER OF COMPUTER APPLICATIONS

(INMCA)

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

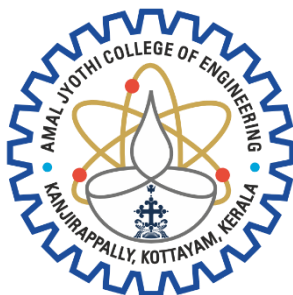


AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS
KANJIRAPPALLY

[Affiliated to APJ Abdul Kalam Technological University, Kerala. Approved by AICTE,
Accredited by NAAC. Koovappally, Kanjirappally, Kottayam, Kerala – 686518]

2020-2025

DEPARTMENT OF COMPUTER APPLICATIONS
AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS
KANJIRAPPALLY



CERTIFICATE

This is to certify that the Project report titled “**FREELANCEHUB**” is the bona fide work of **VARSHA SHAJI (Regno: AJC20MCA-I056)** carried out in partial fulfillment of the requirements for the award of the **Degree of Integrated Master of Computer Applications** at **Amal Jyothi College of Engineering Autonomous, Kanjirappally**, Affiliated to **APJ Abdul Kalam Technological University**. The project was undertaken during the period from **January 01, 2025** to **April 23, 2025**.

Ms. Meera Rose Mathew
Internal Guide

Ms. Meera Rose Mathew
Coordinator

Rev. Fr. Dr. Rubin Thottupurathu Jose
Head of the Department

External Examiner



AMAL JYOTHI
COLLEGE OF ENGINEERING
A U T O N O M O U S
KANJIRAPPALLY



Department of Computer Applications
CERTIFICATE ON PLAGIARISM CHECK

1	Name of the Scholar	
2	Title of the Publication	
3	Name of the Guide	
4	Similar Content (%) identified	
5	Acceptable Maximum Limit	25%
6	Software Used	TURNITIN
8	No.of pages verified	

*Report on plagiarism check result with % of similarity shall be attached.

Name & Signature of the Scholar:

Name & Signature of the Guide:

Name & Signature of the Head of the Department:

Dept. Seal

DECLARATION

I hereby declare that the project report “**FREELANCEHUB**” is a bona fide work done at **Amal Jyothi College of Engineering Autonomous, Kanjirappally**, Affiliated to **APJ Abdul Kalam Technological University**, towards the partial fulfilment of the requirements for the award of the **Integrated Master of Computer Applications (INMCA)** during the period from **January 01, 2025** to **April 23, 2025**.

Date:
KANJIRAPPALLY

VARSHA SHAJI
Reg: AJC20MCA-I056

ACKNOWLEDGEMENT

First and foremost, I thank God almighty for his eternal love and protection throughout the project. I take this opportunity to express my gratitude to all who helped me in completing this project successfully. It has been said that gratitude is the memory of the heart. I wish to express my sincere gratitude to our Director (Administration) **Rev. Fr. Dr. Roy Abraham Pazhayaparampil** and Principal **Dr. Lillykutty Jacob** for providing good faculty for guidance.

I owe a great depth of gratitude towards our Head of the Department **Rev.Fr.Dr. Rubin Thottupurathu Jose** for helping us. I extend my whole hearted thanks to the project coordinator **Ms. Meera Rose Mathew** for her valuable suggestions and for overwhelming concern and guidance from the beginning to the end of the project. I would also express sincere gratitude to my guide **Ms. Meera Rose Mathew** for her inspiration and helping hand.

I thank our beloved teachers for their cooperation and suggestions that helped me throughout the project. I express my thanks to all my friends and classmates for their interest, dedication, and encouragement shown towards the project. I convey my hearty thanks to my family for the moral support, suggestions, and encouragement to make this venture a success.

VARSHA SHAJI

ABSTRACT

FreelanceHub is a customized web-based freelance job managing system focused on enhancing collaboration between clients and freelancers. It allows posting and managing projects to be completed in Scrum framework. Additional, chat to be had in real-time, smart suggestions capabilities, conversation through AI technologies, and real time skill deficiency analysis with respective educational recommendations functions are accessible.

FreelanceHub enhances the way freelancers interact with the clients by giving them the opportunity to create rich profiles, join preconfigured teams with designated roles, and work together on projects. Clients can organize and advertise contests, issue diploma, and employ freelancers as workers. Trust is built by a compliment and rating system along with accountability. Payment through installments is made easier and secure.

FreelanceHub provides enhanced administrative features, enabling contest organization and user management while facilitating the resolution of disputes between clients and admins. Enhanced access is granted to those willing to subscribe to advanced features. Eliminate inefficiencies by blending thoughtful design, smart tools and collaborative workflows around the user. Secure Freelance working with clients professionally and engage in an enjoyable experience.

CONTENT

SL. NO	TOPIC	PAGE NO
1	INTRODUCTION	01
1.1	PROJECT OVERVIEW	02
1.2	PROJECT SPECIFICATION	02
2	SYSTEM STUDY	04
2.1	INTRODUCTION	05
2.2	EXISTING SYSTEM	05
2.2.1	DRAWBACKS OF EXISTING SYSTEM	06
2.3	PROPOSED SYSTEM	06
2.3.1	ADVANTAGES OF PROPOSED SYSTEM	06
3	REQUIREMENT ANALYSIS	07
3.1	FEASIBILITY STUDY	08
3.1.1	ECONOMICAL FEASIBILITY	08
3.1.2	TECHNICAL FEASIBILITY	08
3.1.3	BEHAVIORAL FEASIBILITY	08
3.1.4	FEASIBILITY STUDY QUESTIONNAIRE	09
3.1.5	GEOTAGGED PHOTOGRAPH	15
3.2	SYSTEM SPECIFICATION	15
3.2.1	HARDWARE SPECIFICATION	15
3.2.2	SOFTWARE SPECIFICATION	15
3.3	SOFTWARE DESCRIPTION	16
3.3.1	DJANGO	16
3.3.2	MARIADB	16
4	SYSTEM DESIGN	17
4.1	INTRODUCTION	18
4.2	UML DIAGRAM	18
4.2.1	USE CASE DIAGRAM	18
4.2.2	SEQUENCE DIAGRAM	19
4.2.3	STATE CHART DIAGRAM	21
4.2.4	ACTIVITY DIAGRAM	
4.2.5	CLASS DIAGRAM	
4.2.6	OBJECT DIAGRAM	

4.2.7	COMPONENT DIAGRAM	
4.2.8	DEPLOYMENT DIAGRAM	
4.3	USER INTERFACE DESIGN USING FIGMA	
4.4	DATABASE DESIGN	
4.4.1	RDBMS	
4.4.2	NORMALIZATION	
4.4.3	SANITIZATION	
4.4.4	INDEXING	
4.5	TABLE DESIGN	
5	SYSTEM TESTING	
5.1	INTRODUCTION	
5.2	TEST PLAN	
5.2.1	UNIT TESTING	
5.2.2	INTEGRATION TESTING	
5.2.3	VALIDATION TESTING	
5.2.4	USER ACCEPTANCE TESTING	
5.2.5	AUTOMATION TESTING	
5.2.6	SELENIUM TESTING	
6	IMPLEMENTATION	
6.1	INTRODUCTION	
6.2	IMPLEMENTATION PROCEDURE	
6.2.1	USER TRAINING	
6.2.2	TRAINING ON APPLICATION SOFTWARE	
6.2.3	SYSTEM MAINTENANCE	
6.2.4	HOSTING	
7	CONCLUSION & FUTURE SCOPE	
7.1	CONCLUSION	
7.2	FUTURE SCOPE	
8	BIBLIOGRAPHY	
9	APPENDIX	
9.1	SAMPLE CODE	
9.2	SCREEN SHOTS	
9.3	GIT LOG	
9.4	CERTIFICATES	
9.5	PLAGIARISM REPORT	

List of Abbreviations

IDE	-	Integrated Development Environment
HTML	-	Hyper Text Markup Language
CSS	-	Cascading Style Sheet
UML	-	Unified Modeling Language
MVT	-	Model-View-Template
NLP	-	Natural Language Processing
MariaDB	-	Maria Database
JS	-	JavaScript
AI	-	Artificial Intelligence
API	-	Application Programming Interface
SQL	-	Structured Query Language
UI	-	User Interface
UX	-	User Experience
CRUD	-	Create, Read, Update, Delete
JWT	-	JSON Web Token
SEO	-	Search Engine Optimization
SSL	-	Secure Sockets Layer
CDN	-	Content Delivery Network
AJAX	-	Asynchronous JavaScript and XML
DB	-	Database
ORM	-	Object-Relational Mapping
CSRF	-	Cross-Site Request Forgery
WSGI	-	Web Server Gateway Interface
ASGI	-	Asynchronous Server Gateway Interface
CBV	-	Class-Based View
FBV	-	Function-Based View

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

FreelanceHub is a new web based platform that caters to make a better freelancing experience by providing freelance clients and freelancers. Freelancers can search jobs, offer proposals, and work together while clients can post projects, review proposals, assign tasks, and just track progress. It is built on Scrum principles which means managing the workflow and team work is streamlined and everyone has defined roles like, Project Manager, Developer and Designer. It has key features including real time chat, AI chat bot support, smart project matching, skill gap analysis, event hosting, quizzes and issuance of certifications. FreelanceHub guarantees a reliable and scalable solution of modern project management and freelancer hiring with secure payments, transparent feedbacks and robust administrative tools.

1.2 PROJECT SPECIFICATION

FreelanceHub is web based freelance project management platform that allows clients to connect and perform work for freelance professionals in such a way to be efficient, secure and collaborative. It allows posting of projects, managing proposals, managing tasks, team collaboration, with AI support. Project workflows are based on the Scrum principles of transparency and productivity. Django (backend), MariaDB (database), HTML, CSS, and JavaScript are used as the frontend technologies and build the system.

Users in the Project

In Freelance Hub, there are three primary users:

- **Client:** Individuals or organizations posting job opportunities and managing projects.
- **Freelancer:** Independent professionals offering services and delivering project outcomes.
- **Administrator:** Oversees platform operations, manages user interactions, and ensures platform integrity.

Modules

Client Features

- **Profile Management** – Manage personal details and view past project history.
- **Project Posting** – Post detailed project requirements, scope, deadline, and budget.
- **Proposal Management** – View, accept, or reject freelancer proposals.
- **Task Management** – Define and manage tasks or user stories for projects.
- **Progress Tracking** – Monitor freelancer updates and project milestones.
- **Scrum Meeting Reminders** – Set and receive reminders for Scrum meetings.
- **Notifications** – Receive real-time alerts for updates and communication.

- **Chat System** – Communicate directly with freelancers through integrated chat.
- **Installment-Based Payment Processing** – Make secure milestone-based payments to freelancers.
- **Rating and Review** – Give feedback on freelancer performance after project completion.
- **Complaint Reporting** – Report misconduct or issues with freelancers.
- **Event Hosting** – Create and manage platform-wide events and quizzes.
- **Freelancer Hiring** – Search for and hire freelancers based on project needs.
- **Project Recommendations** – Get recommended freelancers matching project requirements.
- **AI Chatbot for FAQs** – Use AI support for quick answers to platform-related queries.
- **Premium Subscription Access** – Unlock additional features after posting three projects.
- **Password Reset** – Securely reset forgotten passwords.
- **Logout** – Safely log out of the platform.

Freelancer Features

- **Profile Management** – Set up and update skills, experience, and portfolio.
- **Project Search & Proposal Submission** – Search for projects and send personalized proposals.
- **To-Do List** – Track personal tasks and project-related deadlines.
- **Daily Progress Updates** – Provide clients with daily work updates.
- **Team Collaboration** – Form or join teams with assigned roles for collaborative work.
- **Rating and Review Clients** – Evaluate and review client interactions.
- **Complaint Reporting** – Report problems or issues with clients.
- **Notifications** – Receive instant alerts on updates and communication.
- **Chat System** – Communicate in real-time with clients via messaging.
- **Project Recommendations** – Get suggested projects based on skills and history.
- **AI Chatbot for FAQs** – Get automated answers to common platform queries.
- **Premium Subscription Access** – Unlock extra features after completing three projects.
- **Password Reset** – Recover access through secure password reset.
- **Logout** – Exit the account securely.

Administrator Features

- **User Management** – Oversee and manage all platform users and permissions.
- **Complaint Handling** – Investigate and resolve client-freelancer disputes.
- **Analytics Dashboard** – View insights on platform usage and performance.
- **Approve/Reject Events and Quizzes** – Moderate and approve hosted events and quizzes.

CHAPTER 2

SYSTEM STUDY

2.1 INTRODUCTION

The structure of FreelanceHub is subjected to a system study to understand how the platform is designed and how the users interact with the platform and the system. It seeks to uncover how perfectly the system addresses the requirements of clients, freelancers and administratives with the help of project posting and proposal managing, along with collaboration. This study deals with the challenges, goals indication and system analysis methods to perform smoothly. The findings will enable the creation of a reliable and end user relevant freelance job management solution.

2.2 EXISTING SYSTEM

Currently, most of the freelance platforms offer basic functionalities such as posting a project, submitting a proposal and a messaging system. But, these tools and features are not integrated project management tools, real time collaboration features, and personalized recommendations. Communication gaps, lack of effective task tracking, and in efficient denouncing of disputes also negatively affect the user's experience. And most of the platforms also do not allow access to the advanced features until it's not clear what is the progression path for the users. On top of that, no team collaboration, no AI support, not a lot of structured workflows such as Scrum are usually lacking or very poorly developed. These limitations depict that there is a lack of a comprehensive system to do the above.

NATURAL SYSTEM STUDIED

Within the natural system, clients and freelancers might meet in the informal networks, on social media or simply through personal referral. Payment and communications are usually through email or messaging apps, and payments are still done manually. Due to task tracking, performance monitoring and accountability being limited, instead of delays, miscommunication, and lack of transparency.

DESIGNED SYSTEM STUDIED

On the other hand, there are some existing platforms meant for freelancers like Upwork, Fiverr, and Freelancer that offer just rudimentary services — anything from projects posting and proposal submission; that messaging. However, these are the systems which provide the opportunity of online freelancing, but mostly these systems do not provide a good advanced project management, AI support, real time task tracking etc., facilities. There are many locked premium tiers that have limited free access to many of their essential tools.

2.2.1 DRAWBACKS OF EXISTING SYSTEM

- Limited support for Scrum-based project management and task progress tracking.
- No structured team collaboration or defined roles of any project.
- Absence of intelligent skill gap analysis or personalized project recommendations.
- Lack of AI support for frequent queries that are asked or for system guidance.

2.3 PROPOSED SYSTEM

FreelanceHub is a proposal of full featured web based freelance job management platform for clients and freelance managers to avoid the inessential appointment process. The project execution here is done on Scrum principles, communication is done through real time chat, AI powered chat bots are used for query resolution and finally this uses the installation based payment methods. The system features team collaboration modules, hosting of events and quizzes, evaluation of skill gaps, and project recommendations in accordance with user profiles. Users are also shown other administration tools such as managing users, resolving complaints, and maintaining platform integrity.

2.3.1 ADVANTAGES OF PROPOSED SYSTEM

- A project management based on scrum to manage structured and effective task execution.
- Chat, daily progress updates etc. are real time collaboration tools.
- Team formation with defined roles like Project Manager, Developer, QA, etc.
- AI enabled chatbot to assist users on getting quick answers to FAQs.
- Recommendations of smart project and freelancer based on account data.
- Secure, installment-based payment processing with transparency.
- Event and quiz management with certificate issuance and approval system.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 FEASIBILITY STUDY

The feasibility of an information system deals with the possibility of achieving the proposed system's development and deployment. A feasibility study determining the practicability of the solution involves factors such as resources, cost and benefits that are likely to be incurred. This is mainly because there are several common classes of feasibility that are often taken into consideration when developing a management information system. They are:

- Technical feasibility
- Operational feasibility
- Economic feasibility

3.1.1 Economical Feasibility

Economic feasibility evaluates whether the development and running of the software can be made to yield benefits that can offset the costs of development as well as use. Yes, Freelance Hub can be considered to be economically feasible because of no license and development costs due to use of open- source technologies. New revenue sources will include; subscription services where freelancers and clients will be required to pay some amount after they post three jobs to ensure they pay for more services in any given fiscal year. Outsourcing to the cloud has the advantage of cutting costs on basic infrastructure while making room for growth. Since it does not require a lot of money to start and as the company clearly understands how it can generate revenue, it could be highly likely to become highly profitable in the long-run.

3.1.2 Technical Feasibility

Technical viability looks at whether the present technology, resources, and structures of the firm are capable of meeting project specification before the set point of time or budget. Freelance Hub platform is indeed quite viable when it comes to its technological approach, using Django on the back end, while the front end is coded in HTML, CSS and JavaScript. Consequently, MariaDB 5 provides clients with high online transactional processing and capable of data scalability. The use of Artificial Intelligences in the delivery of chatbots has improved the quality as well as the efficiency of service deliveries to the users. There are objective opportunities for developing integrated technologies that can enable future growth at the same time as integrated efficient development.

3.1.3 Behavioral Feasibility

Operational feasibility refers with the implementation of the developed system in the real world environment within the set business objectives and user specifications using currently available

manpower resources in the organization. Organizationally, Freelance Hub is plausible to operate; this is because it enhances communication between the clients and freelancers. Customers may set up events, book acts, and obtain individualized project suggestions, as well as freelancers receive project ideas and specific recommendations for enhancing their profiles. Real time communicate and the in-app notifications, help the users in keeping them updated and in touch. Event moderation is customizable and can be undertaken by the administrators of the said platform. All these features collectively make it possible for the platform to operate, improve efficiency, and be easily usable after implementation.

3.1.4 Feasibility Study Questionnaire

Project Overview:

The Freelance Hub will resolve the problems related to managing the freelance jobs by introducing a complete web-based solution. It tackles problems like slow job posting, poor communication between client and freelancer, and management of payment and project progress. These improvements are aimed at optimizing user satisfaction, streamlining operations for clients and freelancers, and promoting collaboration within the freelance ecosystem.

System Scope:

It is suggested that The Freelance Hub will be a production-ready implementation. It will include all the basic features needed to effectively manage freelance jobs and will be scalable and reliable enough to handle a growing user base and growing overall transaction volumes over time.

Target Audience:

Users of the Freelance Hub generally consist of:

- **Clients:** Users that are looking for freelancers in all kinds of industries.
- **Freelancers:** Self-employed specialists providing their services and knowledge.
- **Administrators:** Manage user accounts and disputes and maintain the integrity and security of the platform.

Modules:

Nuts and Bolts of the Freelance Hub include:

- **Project Posting and Management:** Freelancers bid on projects posted by clients and both parties manage their project listings and proposals.
- **Proposals and Project Management:** Client post a project and receive freelancer recommendations, while freelancers submit proposals

- **User Profile Management** : Clients and freelancers create and manage their profiles, where premium features are unlocked after posting/completing projects.
- **Real-time Chat System**: The clients and freelancers discuss the projects through chat, video, and audio calls.
- **Payment Processing**: Secure payment transactions are processed, with updates and reminders sent for client and freelancers.
- **Rating and Review System**: Freelancers are rated by clients for performance which aids in increasing trust and accountability.
- **Admin Dashboard**: Create and Manage user accounts, Monitor User Activity, Fraud Detection, Approve/Reject events.
- **FAQs AI ChatBot**: Both clients and freelancers can get instant replies to any queries on the platform here, thanks to an AI chatbot.
- **Events Creation**: customers can create events and quizzes then give prizes.

User Roles:

FreelanceHub user roles are defined as:

- **Client**: Can create projects, manage proposals, and communicate with freelancers, see the work progress, release payment, and give feedback.
- **Freelancer**: Search for projects, submit proposals, communicate with clients, manage tasks and milestones, receive pay, and build their reputation through ratings and reviews from clients.
- **Administrator**: Manages the user account, oversees the operation of the whole platform, deals with disputes, ensures compliance with platform policies, optimizes for user experience.

System Ownership:

The Freelance Hub is developed by an academic institution for a project purpose.

Industry/Domain:

The Freelance Hub works in the freelancing job management sector, serving multiple industries that require freelance services, such as technology, creative arts, and others.

Data Collection Contacts:

Name: Linoy Stephen

Role: Director

Contact information: 9633200034

Company: Nanobird Technologies

Questionnaire for Data Collection:**1. What is the most popular freelance jobs that are being requested at this time?**

Sales, billing, accounting, web development, graphic design, content writing and digital marketing are the most popular freelance jobs.

2. How do clients normally decide which freelancer should work for their projects?

This means clients usually choose a freelancer relying on resume, related experiences, portfolio, reviews and rating from added clients as well as proposal.

3. When the freelancers search for freelance job opportunities what do they usually look for?

They are freelancers who try to search for jobs they would be eligible for using their skills, experience, pay they would be getting, and clear project descriptions. At the same time, they enjoy clients who have a history of payments on time and positive feedback of other freelancers.

4. What is the preferred way freelancers communicate to clients throughout the work process?

The easiest and the best way to communicate for freelance people is by using integrated real time chat system inside the platform. Additionally, they use email and video calls for broader discussions and project updates.

5. How is it usually done, payment between clients and freelancers?

Usually, the payments are taken care of through secure payment gateways like Stripe or PayPal. The payment by clients mostly involves paying a percent upfront and the rest is paid at the completion of the project.

6. How do you think freelancers find most useful features or tools available on freelance job platforms in your experience?

The most useful features include specific job postings, effective proposal management, instant chat system, secure handling of payment, as well as its rating and review system for freelancers.

7. Is the evaluation of clients and freelancers decisive in the field of freelance work?

Feedbacks and ratings are significant since they increase the level of reliability. It would be noted that freelance workers with high rankings enjoy better prospects of being hired because most clients are more inclined to consult services that possess good ratings because ratings act as indicators to the efficiency and credibility of freelancers.

8. It is crucial to know some vital characteristics that may enable the freelance job platform to support users in finding the most suitable employers and effective job searches.

Some of the necessities are as follows: a search system, profile management system, security measures for payment gateway, a rating and review system, real-time communication methods, among others.

9. How do clients and freelancers usually resolve disputes or issues?

Disputes are typically resolved through the platform's resolution center, where both parties can present their case. Administrators or mediators then review the information and make a decision.

10. Which of the following features should be included for communication in the platform, that contains chatting, e-mail, and video conferencing?

However, real-time chat has the potential to bring this convenience of communication, you may further incorporate an email for giving more formal notification or video call option for getting a clear vision of the projects, in a discussion with the clientele or even the freelancers.

11. When it comes to the dissemination of the updates and milestones in the project, how should the freelancers be communicating this to the clients?

One of the issues involves realizing timely project updates and milestones. We recommend:

- Focused Corporate/Project Specific Instrumentation: It is possible to introduce the concepts of Corporate/Project-specific dashboard/boards to track the progress, calendar, and timelines.
- Reports: Automatic creation of progress reports and reports with progress bar and accomplishments.
- Notification: This involves making notification within the application regarding important events within the system, deadlines, completed activities among others.

12. Which of these methods of user verification are appropriate (email, phone, face verification)?

Phone OTP and email verification is enough to ensure adequate security, but it is recommended to add an ID check as well. Additional security is possible by incorporating face verification in the function.

13. Which premium features to include (e.g., boosting profile, unlimited proposals)?

Scalable features provide motivation for users, and premium features provide additional

capabilities. Consider these options:

- **Profile Expansion:** Provide extra profile visibility to attract clients to freelancers.
- **Unlimited Proposals:** Give freelancers the ability to submit unlimited proposals to boost their chances of securing work.
- **Priority Support:** Discerning support channels with a lesser hold up time for premium customers.
- **Individual Advanced Analytics:** Provide in-depth analytics on user activity and project insights.

14. Will the platform take a cut for each job posting, transaction, or membership?

For a platform to be sustainable, it needs to generate revenue. Consider the following fee structures:

- **Project Posting Fees** — Charge clients fees to post jobs.
- **Commission Fees:** Take a tiny percentage on successful transactions between projects.
- **Membership Fees:** Introduce tiered membership plans offering different levels of benefits and features.

15. What notifications do we want users to receive (project updates, payment reminders, etc.)

Notification should cover: Project updates and Milestones, Payment reminders and successful payments, Proposal acceptance or rejection New job postings based on preferences.

16. What notification types (email or in-app or WhatsApp, etc.) should use?

Distribute notifications through several mediums as per user preference:

- **In-app Notifications:** For real time and eye-catching notifications.
- **Email Notifications:** More formal full length update.

17. Will the platform be a site for hosting events like webinars, quizzes, or networking sessions to prompt engagement?

Yes, hosting webinars, workshops, virtual meet-ups, and events to engage users to provide value-added services.

18. How would you manage team collaboration for a project on the platform and which roles would you include (manager, developer, designer etc.)? What particular duties should each role be responsible for as well?

When dealing with complex projects, a crucial aspect is its collaboration features. Consider the following:

- **Project Management Tools:** Link project management tools like Trello or Asana to set out tasks, assign responsibilities, and determine progress.
- **Team Roles:** Specify roles such as Project Manager, Team Lead, Developer, Designer, Quality Assurance for clear responsibilities.
- **Assign and Track Tasks:** Empower project managers to allocate tasks to team members, monitor progress, and give feedback.
- **Communication channels:** Prevent noise with dedicated channels for each team and/or project.

19. Is it possible for clients to hire freelancers directly on the platform?

Yes, the clients will also have the option to hire freelancers directly from their profile or via the proposals they submit for the job postings.

20. Should AI chatbots help users with common questions and platform navigation?

Yes, AI chatbots can help users find answers for commonly asked questions, help users explore the platform, and even provide technical assistance.

21. What new technologies (e.g., AI, machine learning) could enhance platform capabilities?

- **AI-driven Skill Matching:** Include AI capable of matching freelancers to client projects in a better dimension due to experience, skills, and project requirements.
- **Predictive Analytics:** Use machine learning for predicting project timelines; analyzing potential risks; suggesting optimal project workflows;
- **Fraud Detection:** Use AI algorithms to identify fraudulent activities, such as fake profiles or disputes over payments.
- **Personalized Recommendations:** Use AI to suggest the most appropriate freelancers, projects, and training materials to users.

22. Do you have any unique features you want to incorporate that are not usually found on like platforms?

- **Skill Assessment Tests** — Implement skill assessment tests to validate freelancer skills and offer unbiased performance evaluations.
- **Portfolio Showcase:** Allow freelancers to present their work in an attractive and interactive way.
- **Client Reviews and Testimonials:** Develop a strong mechanism to allow clients to review and leave testimonials for freelancers, to foster confidence and credibility.
- **Mentorship Program** — Create a mentorship program that actually connects experienced freelancers with aspiring ones.

23. Should the platform provide training or other resources to help freelancers improve their skills?

Yes, adding skill development courses, webinars and certifications will be value addition to the platform and empower the freelancers.

3.1.5 Geotagged Photograph



3.1 SYSTEM SPECIFICATION

3.2.1 Hardware Specification

- Processor - Min Intel core i5
- RAM - Min 8GB
- Hard Disk - Min 500GB SSD

3.2.2 Software Specification

- Front End - HTML, CSS, JavaScript, Bootstrap, jQuery, AJAX
- Back End - Django
- Database - MariaDB
- Client on PC - Windows 7 and above
- Technologies used - HTML, CSS, JavaScript, Django, MariaDB, AJAX, NLP, Google Gemini API, Email API, Razor pay API, Dynamic Portfolio Creation from resume, Course Recommendations based on skill gap

3.3 SOFTWARE DESCRIPTION

3.3.1 Django

Django is a high-level Python web framework that enables rapid development of secure and scalable web applications. Django functions with the Model-View-Template (MVT) architectural design and incorporates default capabilities for security authentication together with database control and URL routing that enhance coding speed. Django implements a reusable design that reduces duplicate code and follows the implementation standard of "Don't Repeat Yourself" (DRY). Web application developers frequently select this framework to build durable web systems and it can link to multiple database systems.

3.3.2 MariaDB

MariaDB is an open-source relational database management system (RDBMS) that originated as a fork of MySQL. It is known for its reliability, performance, and scalability, making it a popular choice for web applications. MariaDB supports SQL for database queries and offers additional features, improved security, and performance optimizations compared to MySQL. It is fully compatible with MySQL, allowing seamless migration between the two. MariaDB is commonly used with frameworks like Django to manage and store large volumes of structured data.

CHAPTER 4

SYSTEM DESIGN

4.1 INTRODUCTION

In system design, a deep outline of a system's architecture, components, and data flow along with other elements is created to fulfill prescribed functional and non-functional requirements. It deals with its performance, scalability, security, usability, and maintainability. A system's design should be such that it works with optimal efficiency and can be modified easily in the future. UML (Unified Modeling Language) diagrams such as use case, class and sequence diagrams, and activity diagrams are commonly utilized to show the interaction diagrams and structures of a system or workflow. Along with these diagrams, ER diagrams aid in data visualization, and together they help data developers better communicate their design during the design stage.

4.2 UML DIAGRAM

A UML diagram encapsulates a typical visual representation of a system's organization, interactions, and activities within or across each system. In UML diagrams, classes, objects, and relations may be presented with certain shapes like circles and rectangles. Developers, software analysts and other stakeholders can use these diagrams for documenting, building, designing and even reasoning about softwares. These diagrams are versatile and assist in almost every activity during the software development lifecycle. This adaptive quality guarantees the meeting of user needs and requirements during system development. From simple applications to complex enterprise systems, everything can be modeled using UML diagrams.

4.2.1 USE CASE DIAGRAM

The use case diagram illustrates system functionality through its representation of the interactions between users (actors) and the system (use cases). It defines system boundaries which help emphasize essential features and enables early user requirement acquisition throughout development stages to guarantee system goal achievement.

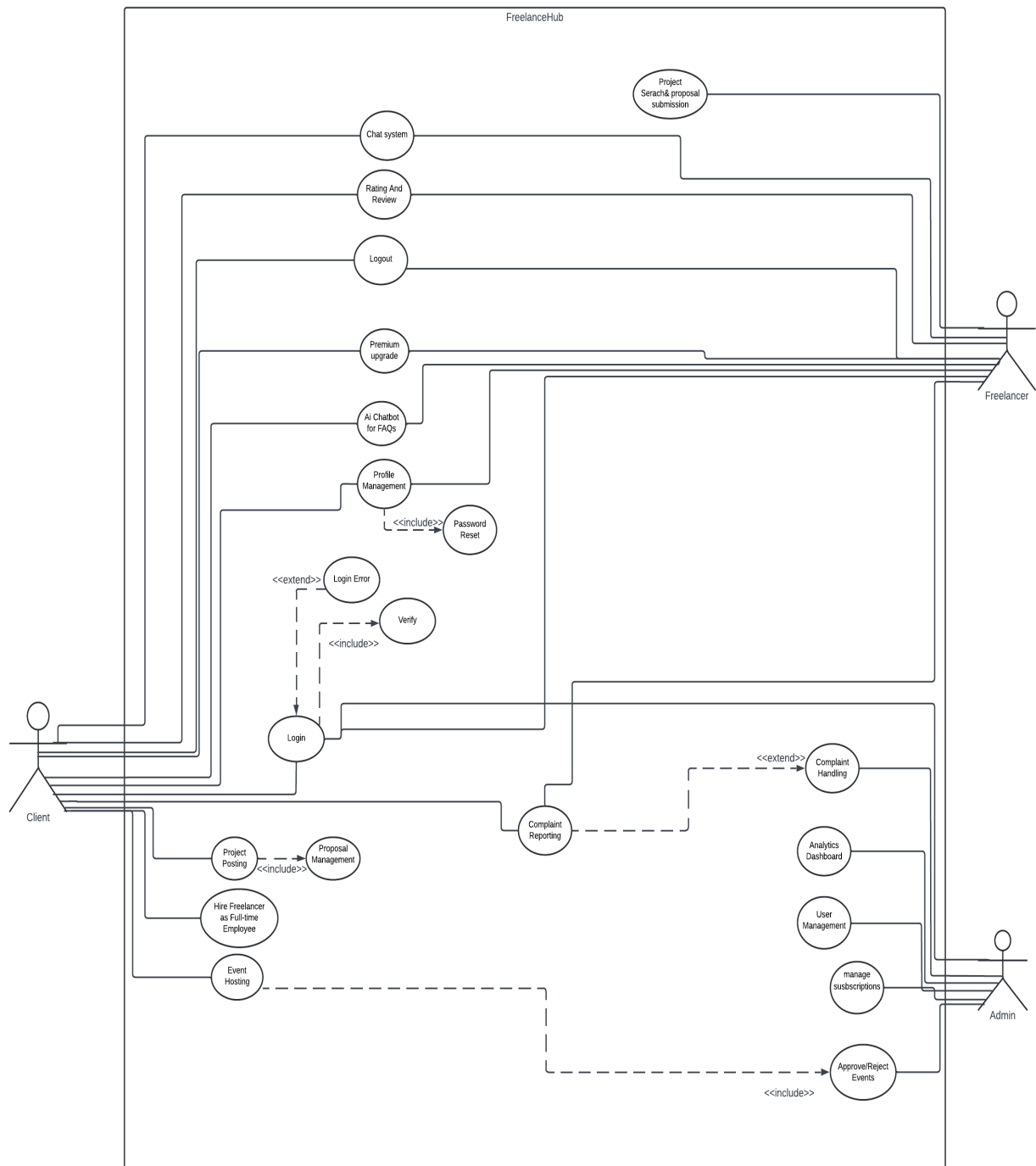


Figure 1: Use case diagram

4.2.2 SEQUENCE DIAGRAM

The sequence diagram functions as an interaction diagram which illustrates object and component communication during a scenario. Vertical lifelines represent objects on the diagram and horizontal arrows symbolize the message transfers between these objects. The display of control flow enables developers to improve software design along with system interaction analysis and produce development documentation.



Figure 2: Sequence diagram

4.2.3 State Chart Diagram

In UML the State Chart Diagram or State Machine Diagram represents an object along with its multiple states and trigger conditions these states use to change. Through State Chart Diagrams we can identify how systems operate dynamically because the diagrams track objects through their operational states. Rounded rectangles define the states and arrows between these states illustrate state transitions and events determine shifts between states and the diagram includes initial and terminating states. Real-time systems and embedded systems use state chart diagrams for model developments in addition to their use in workflow modeling.

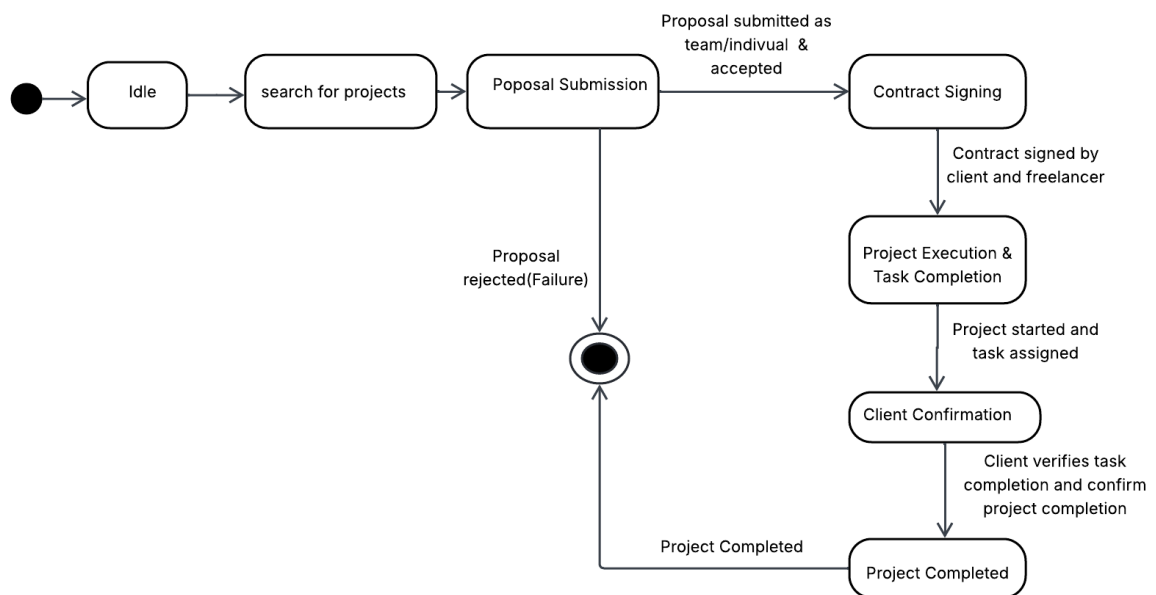


Figure 3: State chart diagram

4.2.4 Activity Diagram

Activity diagrams enable visual workflow modeling through which data and control flow can be represented to explain process behavior. The diagram displays system and use case activities as well as decisions along with their execution order. Subsequent to programming and business process transformation activity diagrams represent complex workflows which allow stakeholders to understand system behavior and processes better.

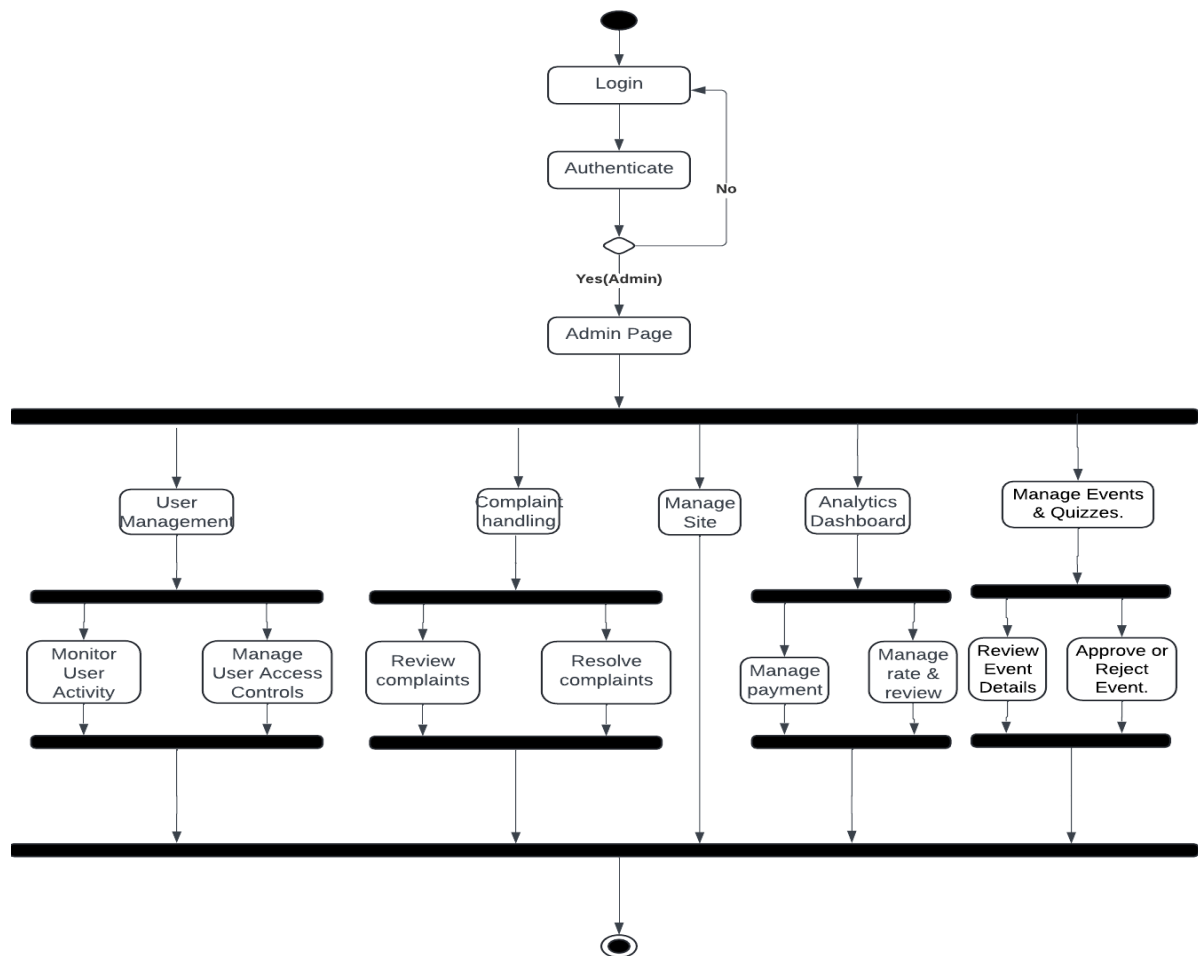


Figure 4: Activity diagram of admin

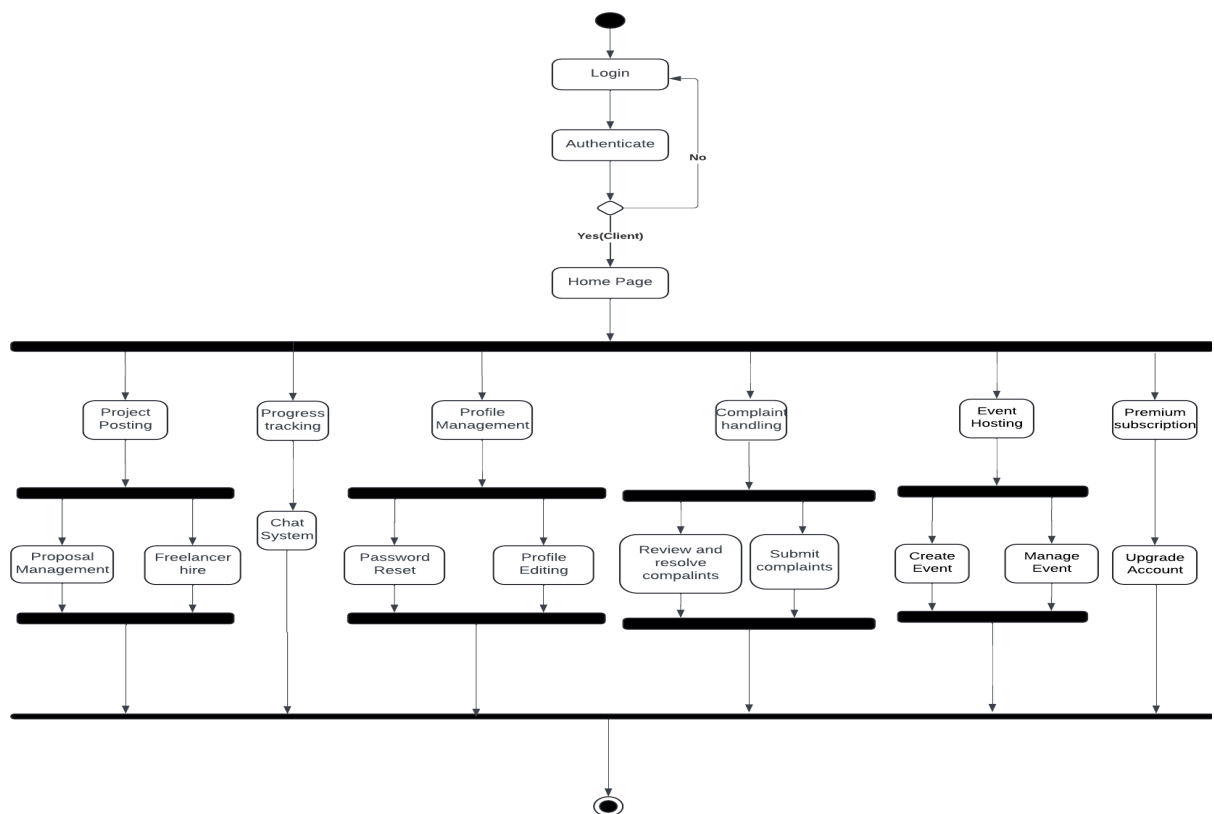


Figure 5: Activity diagram of client

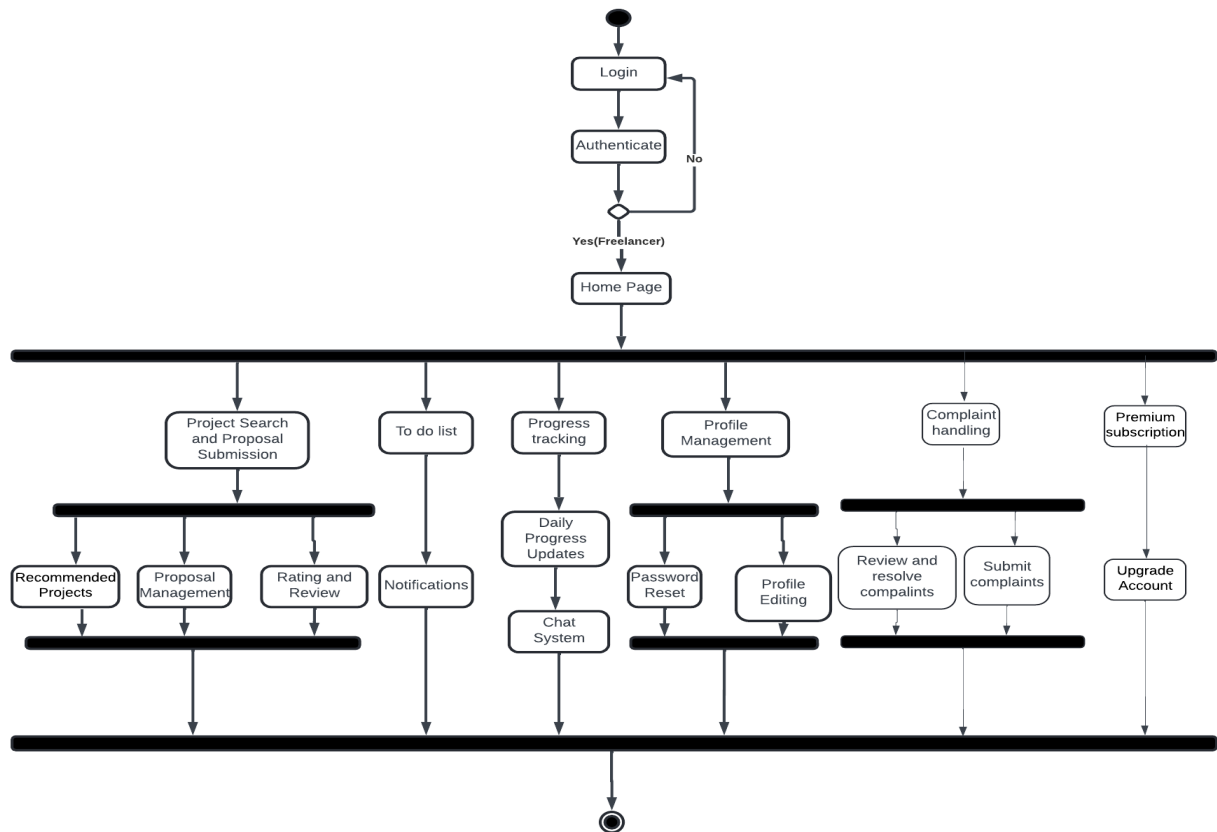


Figure 6: Activity diagram of freelancer

4.2.5 Class Diagram

The object-oriented programming framework uses class diagrams to represent system structures with their classes alongside their attributes and methods and relationships. The diagram helps developers create system architecture designs while describing class interaction patterns and defining essential properties with their associated behaviors. The development process relies on class diagrams as essential tools which maintain developer-stakeholder-team collaboration from planning to communication throughout project development.

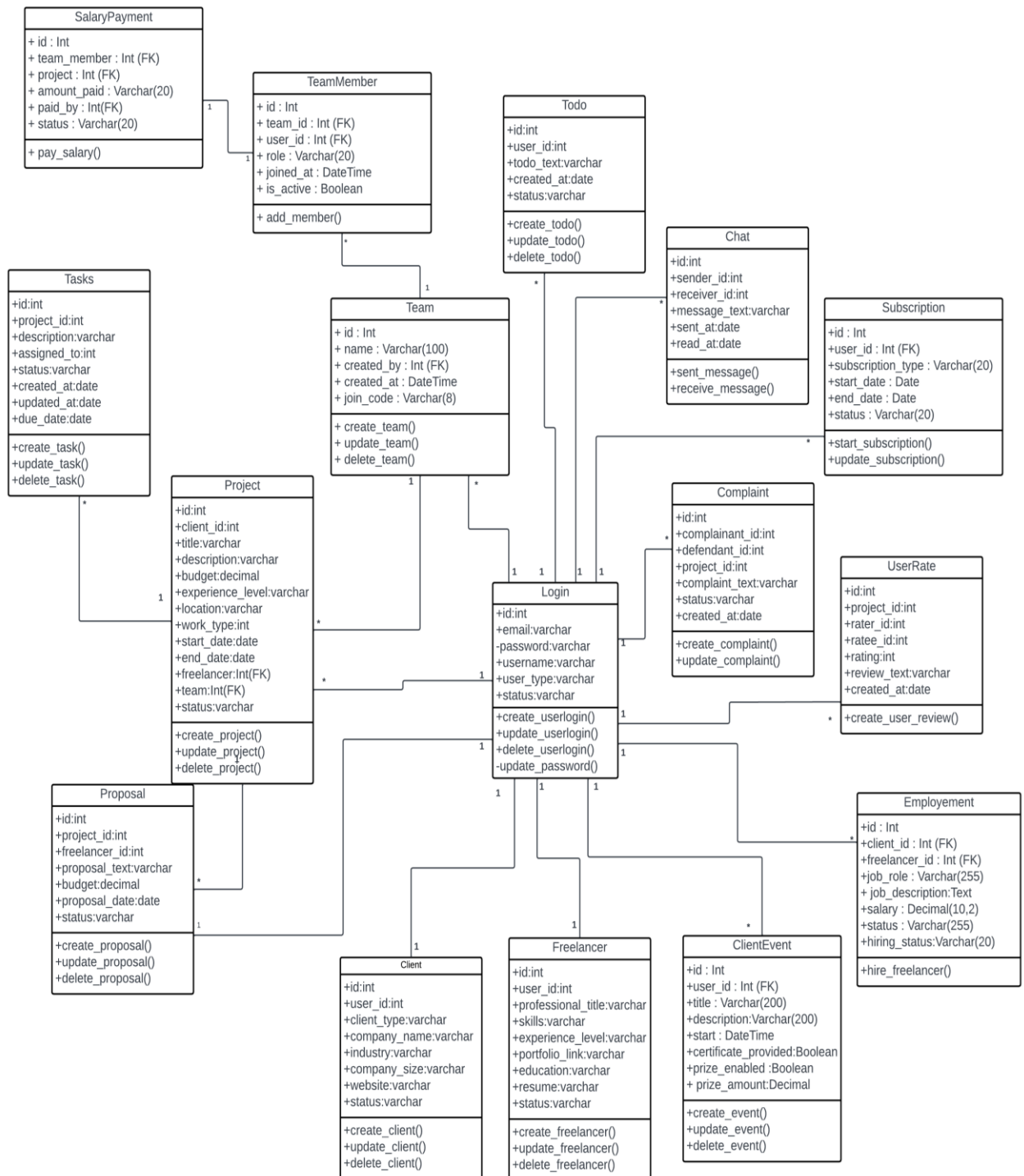


Figure 7: Class diagram

4.2.6 Object Diagram

The UML diagram of object model displays system objects together with their exact relationship bindings at one execution instant. The diagram depicts real-world classes through their objects together with their attribute values as well as object-to-object relationship links. The behavior of a system during execution becomes visible through object diagrams so developers can utilize them for validating class structures and real-time interactions while understanding and analyzing the system.

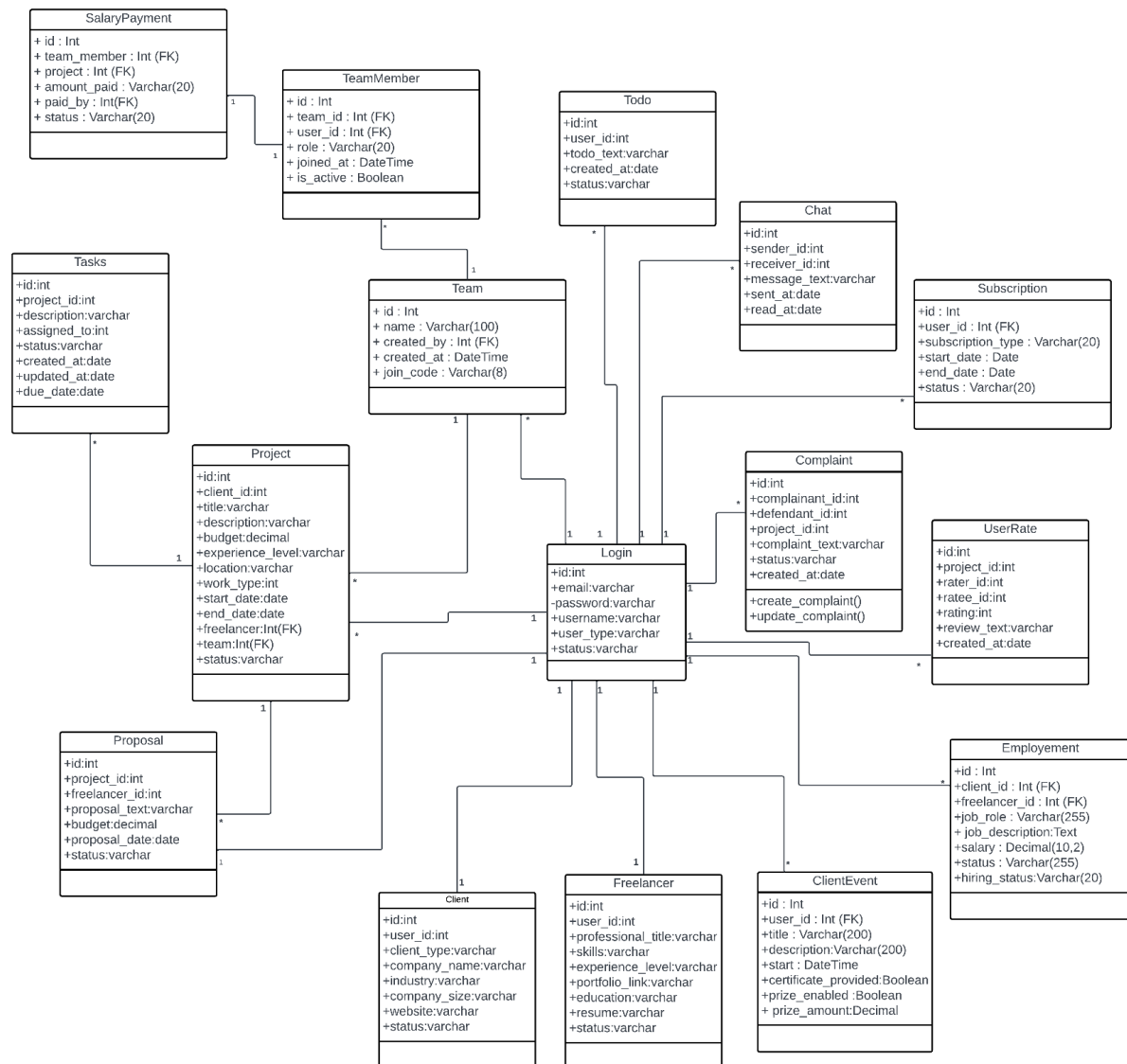


Figure 8: Object diagram

4.2.7 Component Diagram

The UML diagram of Component Diagram depicts system structure through blocks which depict software components with interfaces along with their dependencies. The diagram enables engineers to understand system interaction patterns between modules and APIs and external systems thus supporting the development of modular maintainable software designs. Each component becomes a rectangular shape with connecting interfaces that shows developers how various parts interact and organize within the system design.

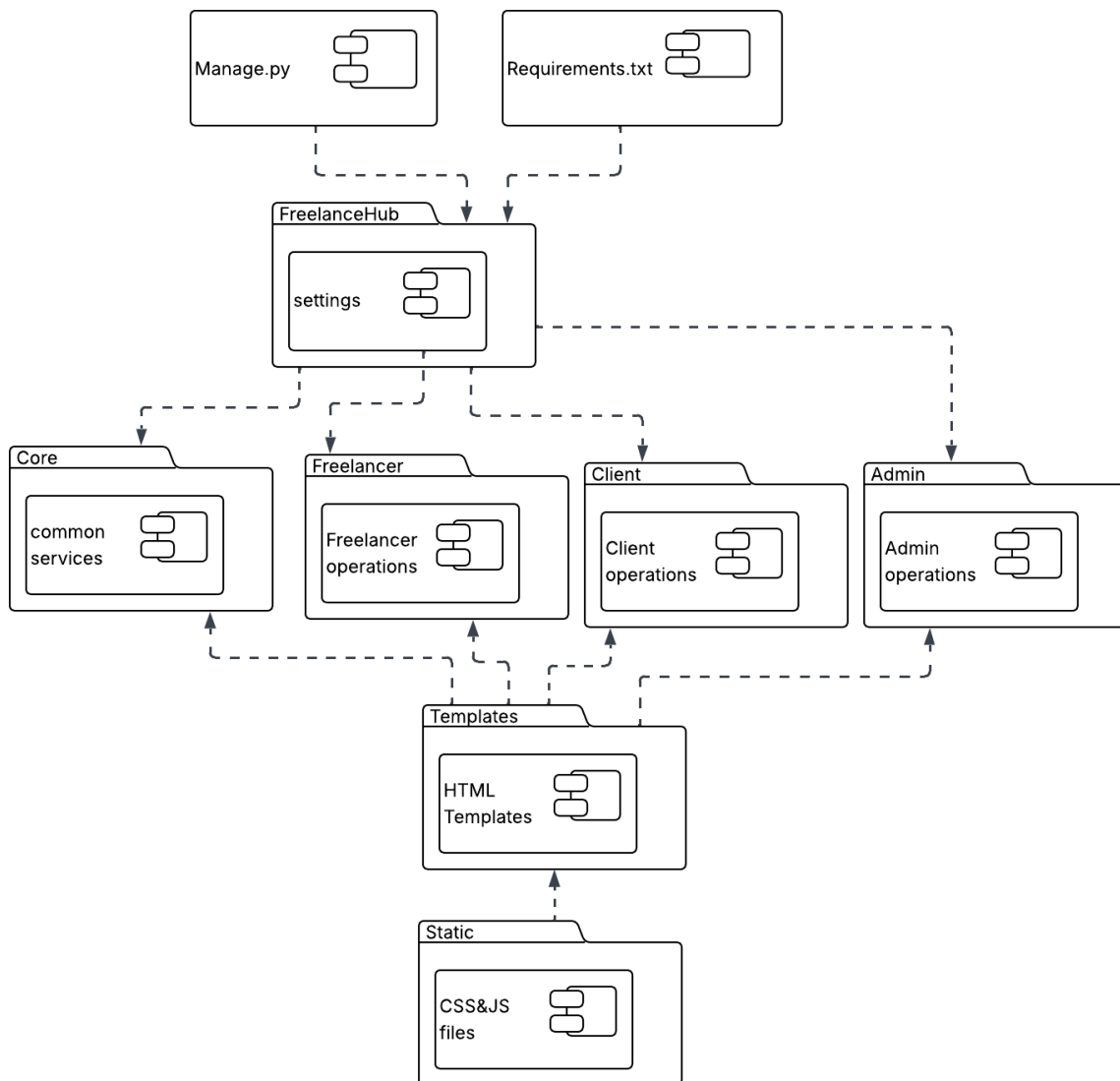


Figure 9: Component diagram

4.2.8 Deployment Diagram

A physical architecture of the system, how the system takes software components to hardware nodes (servers, devices or cloud environment) is represented in a Deployment Diagram using UML. The relationships between the nodes (hardware), artifacts (software components), and communication paths are depicted through it. It aids in system architects as well as DevOps teams' understanding of the system's infrastructure, network distribution, and the deployment strategy.

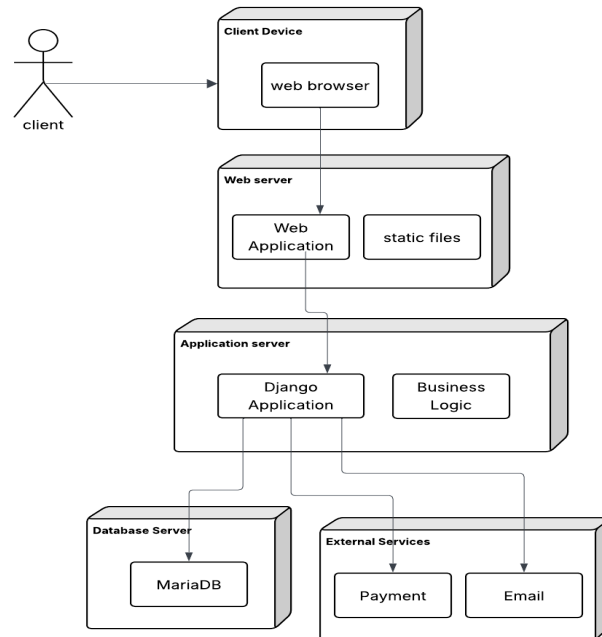


Figure 10: Deployment diagram

4.3 USER INTERFACE DESIGN USING FIGMA

Form Name: Login

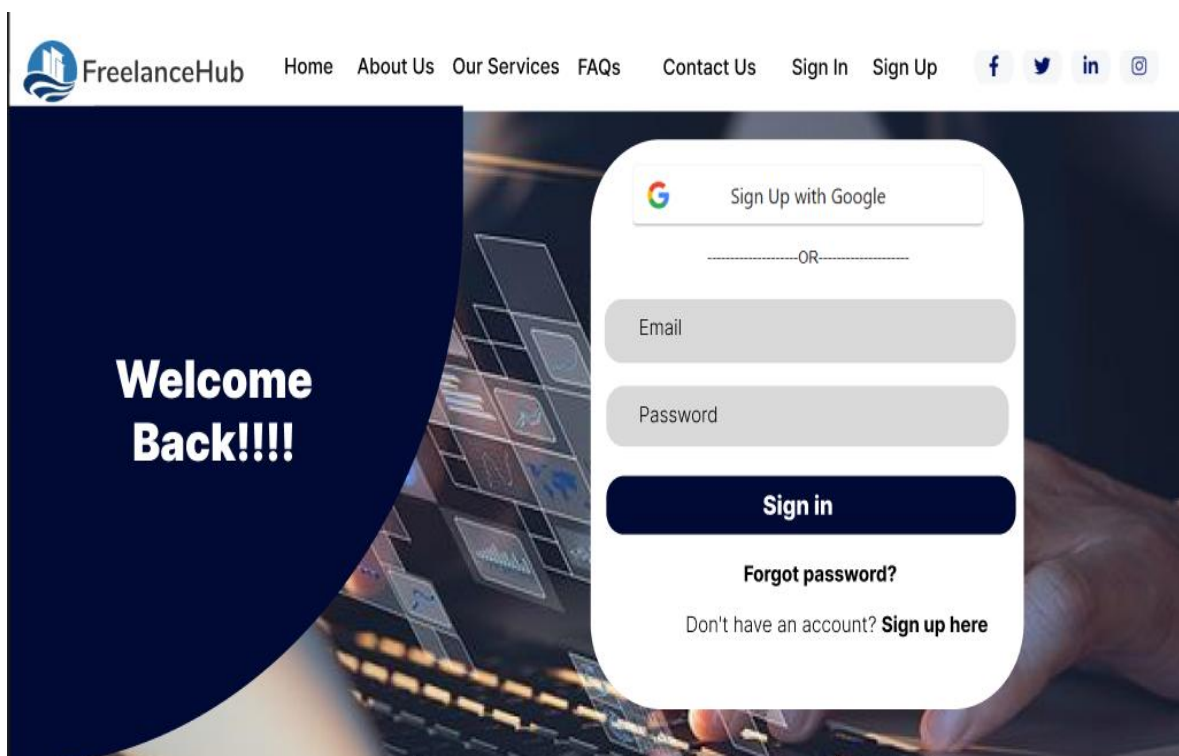
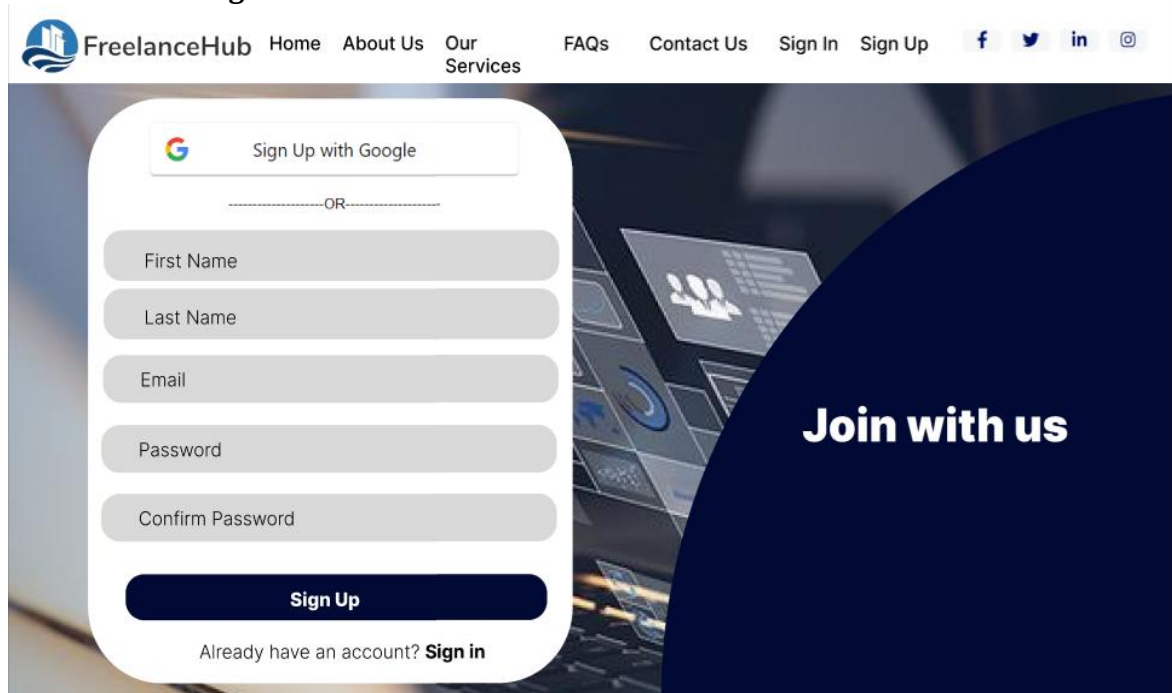
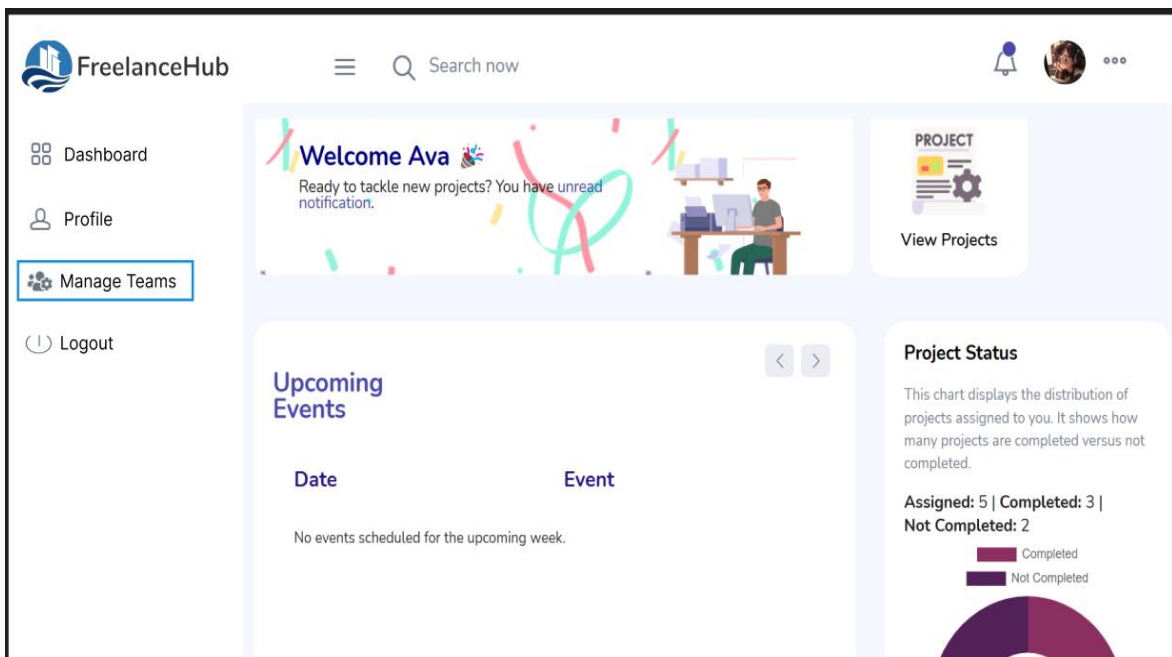


Figure 11: Login page

Form Name: Register

The image shows the 'Register' page of the FreelanceHub website. The page has a dark blue background with a large, semi-transparent white registration form in the center. The form includes a 'Sign Up with Google' button at the top, followed by a separator line with 'OR' in the middle. Below this are five input fields: 'First Name', 'Last Name', 'Email', 'Password', and 'Confirm Password'. At the bottom of the form is a dark blue 'Sign Up' button. Below the button, there is a link that says 'Already have an account? Sign in'. The top of the page features a navigation bar with the FreelanceHub logo, links for 'Home', 'About Us', 'Our Services', 'FAQs', 'Contact Us', 'Sign In', and 'Sign Up', and social media icons for Facebook, Twitter, LinkedIn, and Instagram. On the right side of the background, there is a large white text overlay that says 'Join with us'.

Figure 12: Register Page

Form Name: Freelancer Dashboard

The image shows the 'Freelancer Dashboard' of the FreelanceHub website. The dashboard has a light blue background. On the left side, there is a sidebar with a menu containing 'Dashboard', 'Profile', 'Manage Teams' (which is highlighted with a blue border), and 'Logout'. The main content area is divided into several sections. At the top, there is a 'Welcome Ava' message with a small icon and a notification: 'Ready to tackle new projects? You have unread notification.' To the right of this is a 'PROJECT' section with a 'View Projects' button. Below the welcome message is an 'Upcoming Events' section with a table header showing 'Date' and 'Event'. The table currently displays 'No events scheduled for the upcoming week.' To the right of the events section is a 'Project Status' section. It contains a text description: 'This chart displays the distribution of projects assigned to you. It shows how many projects are completed versus not completed.' Below this is a summary: 'Assigned: 5 | Completed: 3 | Not Completed: 2'. At the bottom of this section is a donut chart with a legend showing 'Completed' in green and 'Not Completed' in red.

Figure 13: Freelancer's Dashboard

Form Name: All Team management Page

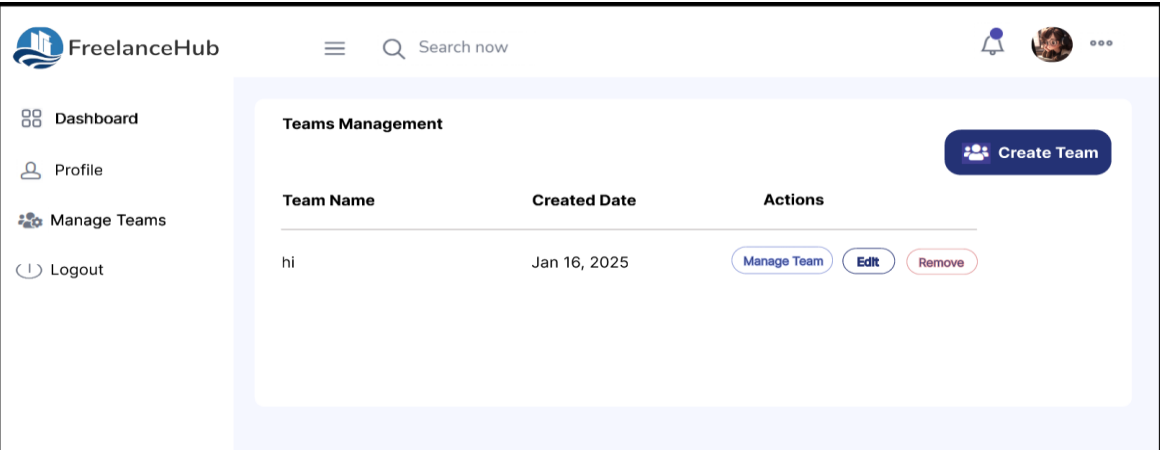


Figure 14: Teams management page

Form Name: New team creation

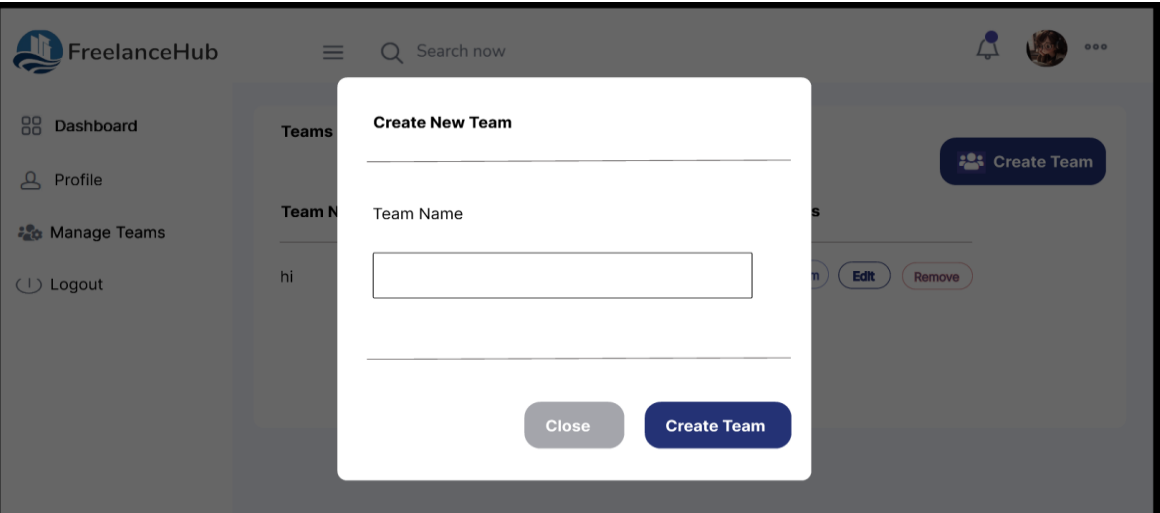


Figure 15: New Team Creation form

Form Name: Team management Page

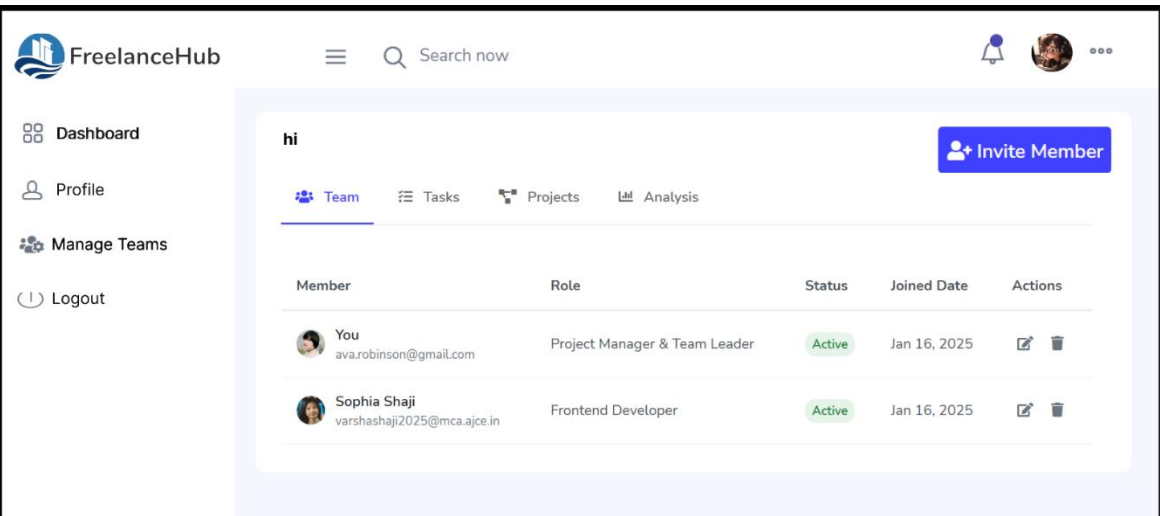


Figure 16: Individual team management page

Form Name: Member Invitation Form

Figure 17: Team member invitation form

4.4 DATABASE DESIGN**4.4.1 Relational Database Management System (RDBMS)**

Database through RDBMS provides storage and administration of structured data in fixed tables that link their data points using specific connections. Tables in an RDBMS maintain distinct fields for data attributes which the SQL language manipulates to execute insertions and updates and make deletions and query operations. Data consistency and integrity in RDBMS function through constraints that include primary keys and foreign keys and unique constraints. Worldwide organizations use this database system to handle structured data volumes efficiently because it provides scalable resources, dependable functionality and methodical data organization.

4.4.2 Normalization

A database organization process named Normalization functions to remove unnecessary data repetitions while enhancing system data quality. Large tables become smaller manageable tables through this approach which also includes defining the relationships between these tables. The database normalization process contains several stages with unique standards which function to eliminate repeated data storage and maintain data unity throughout one storage point. These measures minimize data inconsistencies and anomalies thus supplying better efficiency and accuracy for database operations.

Regularization divides into multiple stages or "normal forms" that define organizational rules for data structures. The most common normal forms are:

- First Normal Form (1NF)

- Second Normal Form (2NF)
- Third Normal Form (3NF)
- Boyce-Codd Normal Form
- Fourth Normal Form
- Fifth normal form

First Normal Form (1NF)

First Normal Form (1NF) exists as the base form of database normalization to enforce a unique primary key and atomic (indivisible) value in all table columns. The First Normal Form ensures data separation into individual tables which eliminates repeated groups while improving database integrity and reducing administrative complexity.

For example:

Relation STUDENT in table 1 is not in 1NF because of multi-valued attribute STUD_PHONE. Its decomposition into 1NF has been shown in table 2.

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY
1	RAM	9716271721, 9871717178	HARYANA	INDIA
2	RAM	9898297281	PUNJAB	INDIA
3	SURESH		PUNJAB	INDIA

Table 1

Conversion to first normal form

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY
1	RAM	9716271721	HARYANA	
1	RAM	9871717178	HARYANA	INDIA
2	RAM	9898297281	PUNJAB	INDIA
3	SURESH		PUNJAB	INDIA

Table 2

Second Normal Form (2NF)

The Second Normal Form (2NF) requires complete dependence of all non-key attributes on full primary keys to minimize table dependencies when multiple attributes make up the primary key. The data maintains higher integrity while it reduces redundancy because the organization method supports easier query execution and simpler maintenance operations.

For Example:

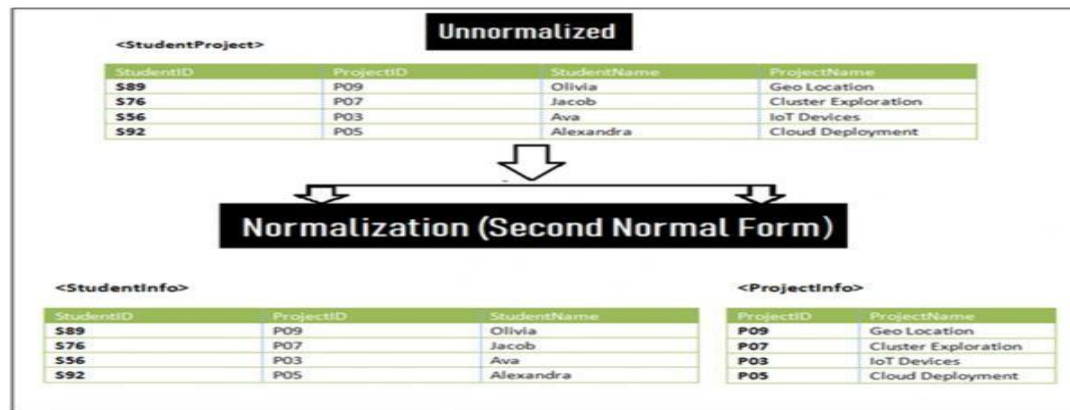
The prime key attributes are **StudentID** and **ProjectID**.

As stated, the non-prime attributes i.e. **StudentName** and **ProjectName** should be functionally dependent on part of a candidate key, to be Partial Dependent.

The **StudentName** can be determined by **StudentID**, which makes the relation Partial Dependent.

The **ProjectName** can be determined by **ProjectID**, which makes the relation Partial Dependent.

Therefore, the <StudentProject> relation violates the 2NF in Normalization and is considered a bad database design.



Third Normal Form (3NF)

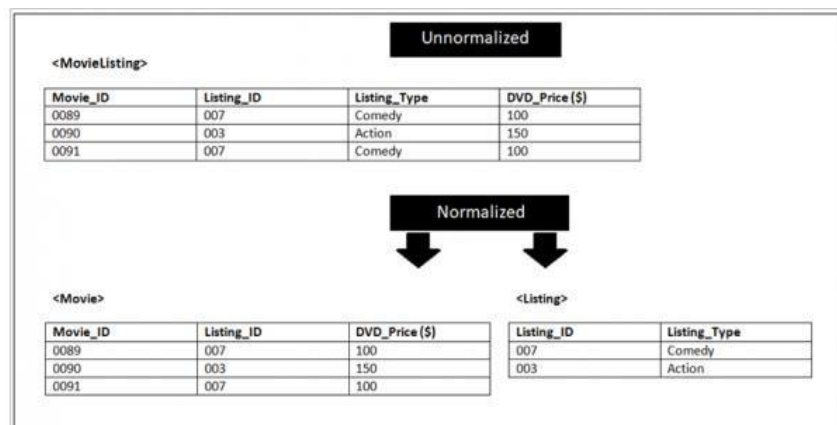
The objective of Third Normal Form (3NF) is to restrict all non-key attributes in a table from depending on anything but the primary key to prevent transitive dependencies. Achieving these data management principles reduces unneeded data duplications and strengthens the overall data reliability. Table splitting occurs as a common operational step when attempting to reach 3NF standards. The database structure may become complex due to this enhancement yet the use case determines if normal form implementation is necessary.

For Example:

The table is not in 3NF because it has a transitive functional dependency –

Movie_ID -> Listing_ID Listing_ID -> Listing_Type

Therefore, **Movie_ID -> Listing_Type** i.e. transitive functional dependency.



Boyce-Codd Normal Form

A database requires every determinant under the BCNF rule to function as a candidate key. Functional dependencies between non-key attributes that are not trivial must be removed to decrease database anomalies and prevent data inconsistencies. The achievement of BCNF standard demands that tables are split which adds complication but delivers superior data consistency while cutting down duplicate storage. The complete benefits become apparent with this method when it

is utilized in extensive database systems since it optimizes query execution and minimizes potential data errors.

The scenario uses relation $R(A, B, C)$ to illustrate this case.

$A \rightarrow BC,$

$B \rightarrow A$

Because keys A and B represent super keys we can confirm that the above mentioned relation fulfills BCNF standards.

Fourth Normal Form

Fourth Normal Form (4NF) advances Third Normal Form (3NF) through its approach to solve multi-valued dependencies inside database structures requiring non-key attributes to functionally depend on the primary key as the only criterion while removing all non-trivial multi-valued dependencies. A multi-valued dependency exists when different independent non-key attributes separate from each other by a primary key.

During the table partitioning process of 4NF each table contains both the primary key and a single dependent attribute. Table breakup during this procedure eradicates repeated data and upholds database consistency while preventing data inconsistency.

A typical case where 4NF application applies to tables is in "Student Course Schedule" information tables. The table named "Course_Schedule" contains three primary key combinations with the following fields:

- Student_ID (Primary Key)
- Course_ID (Primary Key)
- Semester
- Grade

This table enables multiple course enrollments for each student at once along with supporting one course being taught to different students at same time. The relationship between students and courses consists of multiple students taking numerous courses at once. The instructor teaching a course becomes different throughout each semester while the courses themselves show variation based on instructor assignments within a semester.

The relationship between Course_ID and Instructor fields exists in multiple values because of this design. Normalization of the table to 4NF requires splitting it into the two separate entities: Course_Schedule and Course_Instructor.

1. Course_Schedule table:

- Student_ID (Primary Key)

- Course_ID (Primary Key)
- Semester
- Grade

2. Course_Instructor table:

- Course_ID (Primary Key)
- Semester (Primary Key)
- Instructor

Our normalization brought a solution to the multivalued dependency issue while making every entry in our database single-valued without duplication. The independent updates of the Course_Instructor table will maintain dataset consistency with the Course_Schedule table contents.

Fifth normal form

The 5NF (Project-Join Normal Form) which tackles complicated many-to-many connections breaks down tables to delete superfluous data and keeps data consistency. Join operations allow reconstructing tables which leads to better consistency and flexibility. The attainment of 5NF delivers benefits yet its establishment involves substantial complexity together with possible unpractical aspects.

4.4.3 Sanitization

Database security depends on sanitization through the process which cleans problematic inputs thus defending databases from SQL injection and XSS attacks. Database security together with integrity maintenance becomes possible through input validation and parameterized queries and prepared statements. Database systems require proper sanitization methods to create secure dependability.

4.4.4 Indexing

In database structure design indexing creates data organization systems which use column values for quick retrieval by identifying the database locations. The system assists users in finding necessary data rapidly without needing to review entire databases which boosts query execution speeds notably in big or intricately designed databases. Selective usage of indexes remains crucial because multiple indexes negatively affect write operations speed and expand storage needs.

4.5 TABLE DESIGN

1. tbl_customuser

Primary Key: id

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	username	Varchar (128)	Not Null	Username of the user
3	Email	Varchar (255)	Unique,Not Null	Email of the user
4	password	Varchar (128)	Not Null	Password of the user
5	status	Varchar (50)	Default 'active'	User status (active/inactive)
6	joined	DateTime	Auto Add	Date when the user joined
7	role	Varchar (20)	Null	Role of the user (admin, client, freelancer)
8	welcome_email_sent	Boolean	Default False	Whether the welcome email has been sent
9	permission	Boolean	Default False	Permission status
10	email_verified	Boolean	Default False	Email verification status
11	google	Boolean	Default False	Logged in via Google
12	complaint_count	Int	Default 0	Number of complaints against the user

2. tbl_register

Primary key: id

Foreign key: user_id references table tbl_customuser(id)

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	user_id	Int	Foreign Key	Login Id of user
3	first_name	Varchar (150)	Null	First name of the user
4	last_name	Varchar (150)	Null	Last name of the user
5	phone_number	Varchar (10)	Null	Phone number of the user
6	profile_picture	Varchar (255)	Null	Profile picture URL
7	bio_description	Text	Null	Bio description
8	location	Varchar (255)	Null	User's location
9	linkedin	Varchar (255)	Null	LinkedIn profile link

10	instagram	Varchar (255)	Null	Instagram profile link
11	twitter	Varchar (255)	Null	Twitter profile link

3. tbl_passwordreset

Primary key: id

Foreign Key: user_id references table **tbl_customuser(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	user_id	Int	Foreign Key	Login Id of user
3	token	Varchar (50)	Default None	Token for password reset
4	created_at	DateTime	Auto Add	Token creation timestamp
5	expiry	DateTime	Not Null	Expiry date/time of the token

4. tbl_emailverification

Primary key: id

Foreign key: user_id references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	user_id	Int	Foreign Key	Login Id of user
3	token	Varchar (50)	Default None	Token for email verification
4	created_at	DateTime	Auto Add	Token creation timestamp
5	expiry	DateTime	Not Null	Expiry date/time of the token

5. tbl_event

Primary key: id

Foreign Key: user_id references table **tbl_customuser(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	title	Varchar (200)	Not Null	Title of the event
3	start_time	DateTime	Not Null	Event start time
4	end_time	DateTime	Not Null	Event end time

5	description	Text	Null	Description of the event
6	color	Varchar (7)	Default '#ffffff'	Color code of the event
7	user_id	Int	Foreign Key	Login Id of user

6. tbl_notification

Primary key: id

Foreign key: user_id references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	user_id	Int	Foreign Key	Login Id of user
3	message	Varchar (255)	Not Null	Notification message
4	is_read	Boolean	Default False	Indicates if the notification has been read
5	created_at	DateTime	Auto Now Add	Timestamp when the notification was created

7. tbl_site_review

Primary key: id

Foreign key: user_id references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	user_id	Int	Foreign Key	Login Id of user
3	review_text	Text	Not Null	Review text provided by the user
4	rating	Positive Int	Not Null	Rating given by the user
5	created_at	DateTime	Default Now	Timestamp when the review was created

8. tbl_cancellation_request

Primary key: id

Foreign key: project_id references table **tbl_project**, **requested_by** references table **tbl_customuser**, **approver** references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
-----	-----------	-----------------	-----------------	--------------------------

1	id	Int (Auto)	Primary Key	Primary key of the table
2	project_id	Int	Foreign Key	Foreign key referencing the project
3	requested_by	Int	Foreign Key	The user who Requested cancellation
4	approver	Int	Foreign Key	The user who approves the request
5	status	Varchar (10)	Default 'Pending'	Status of the cancellation request
6	requested_date	DateTime	Default Now	Timestamp when the request was made
7	response_date	DateTime	Null	Timestamp when the request was responded to
8	reason	Text	Null	Reason for cancellation

9. tbl_refundpayment

Primary key: id

Foreign key: pay_to references table **tbl_customuser**, **user** references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	amount	Decimal(10, 2)	Not Null	The amount to be refunded
3	pay_to	Int	Foreign Key	Recipient of the refund
4	is_paid	Boolean	Default False	Whether the refund payment has been paid
5	payment_date	DateTime	Null	Date and time when the refund payment was made
6	razorpay_order_id	Varchar(255)	Null	Razorpay order ID for the refund payment
7	razorpay_payment_id	Varchar(255)	Null	Razorpay payment ID for the refund payment
8	user	Int	Foreign Key	User who created the refund payment
9	total_paid	Decimal(10, 2)	Default 0.00	Total amount that has been paid
10	compensation_amount	Decimal(10, 2)	Default 0.00	Compensation amount included in the refund

10. tbl_clientprofile

Primary key: id

Foreign key: user_id references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	user	Int	Foreign Key	Login Id of user
3	client_type	Varchar(50)	Not Null	Type of client ('Individual' or 'Company')
4	company_name	Varchar(255)	Null	Name of the company, if applicable
5	website	Varchar(255)	Null	Website URL of the client
6	license_number	Varchar(255)	Null	License number of the client
7	aadhaar_document	Varchar (file)	Null	Aadhaar document file

11. tbl_repository

Primary key: id

Foreign key: created_by references table **tbl_customuser**, project_id references table **tbl_project**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	project_id	Int	Foreign Key	Project which the repository belongs to
3	name	Varchar(255)	Not Null	Name of the repository
4	created_at	DateTime	Default Now	Timestamp when the repository was created
5	created_by	Int	Foreign Key	Id of user who created the repository

12. tbl_project

Primary key: id

Foreign key: user_id references table **tbl_reg**, freelancer references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	title	Varchar(255)	Null	Title of the project
3	description	Text	Null	Description of the project
4	budget	Int	Null	Budget allocated for the project
5	category	Varchar(605)	Null	Category of the project

6	allow_bid	Boolean	Null	Whether bidding is allowed on the project
7	end_date	Date	Null	End date for the project
8	file_upload	Varchar (file)	Null	File upload for the project
9	created_at	DateTime	Default Now	Timestamp when the project was created
10	User_id	Int	Foreign Key	Id of client
11	status	Varchar(20)	Default:open	Status of the project (open/closed)
12	start_date	Date	Null	Start date of the project
13	project_end_date	Date	Null	End date of the project
14	project_status	Varchar(50)	Default 'Not Started'	Current status of the project
15	freelancer	Int	Foreign Key Null	Id of Freelancer
16	git_repo_link	Varchar(200)	Null	URL to the project's Git repository
17	gst_rate	Decimal (5,2)	Default 18.00	GST rate applied to the project
18	gst_amount	Decimal (15,2)	Default 0.00	GST amount calculated
19	total_including_gst	Decimal (15,2)	Default 0.00	Total amount including GST
20	client_review_given	Boolean	Default False	Whether the client review is given
21	freelancer_review_given	Boolean	Default False	Whether the freelancer review is given
22	scope	Varchar(10)	Default 'medium'	Scope of the project (low/medium/high)
23	Team_id	Int	Foreign Key	References tbl_team(id)
24	required_skills	JSON	Not Null, Default []	Stores skills required for project as a list

13. tbl_sharedfile**Primary key: id****Foreign key: uid** references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	repository	Int	Foreign Key	Id of repository to which file belong to
3	file	Varchar(file)		File uploaded
4	description	Text	Null	Description of the file

5	uploaded_by	Int	Foreign Key	Id of user who uploaded the file
6	uploaded_at	DateTime	Default Now	Timestamp when the file was uploaded

14. tbl_sharedurl**Primary key: id****Foreign key: repository** references table **tbl_repository**, **shared_by** references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	repository	Int	Foreign Key	Id of repository to which url belong to
3	url	Varchar(200)		URL shared within the repository
4	description	Text	Null	Description of the URL
5	shared_by	Int	Foreign Key	Id of user who shared the url
6	shared_at	DateTime	Default Now	Timestamp when the URL was shared

15. tbl_sharednote**Primary key: id****Foreign Key: repository** references table **tbl_repository**, **added_by** references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	repository	Int	Foreign Key	Id of repository to which note belong to
3	note	Text		Note added to the repository
4	added_by	Int	Foreign Key	Id of user who uploaded the note
5	added_at	DateTime	Default Now	Timestamp when the note was added

16. tbl_task**Primary key: id****Foreign key: project** references table **tbl_project**, **assigned_to** references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	project	Int	Foreign Key	Id of project

3	title	Varchar(255)		Title of the task
4	description	Text	Null	Description of the task
5	start_date	Date	Null	Start date of the task
6	due_date	DateTime		Due date of the task
7	status	Varchar(20)	Default 'Pending'	Status of the task ('Pending', 'InProgress', etc.)
8	created_at	DateTime	Default Now	Timestamp when the task Was created
3	title	Varchar(255)	Not null	Title of the task
9	updated_at	DateTime	Default Now	Timestamp when the task was Last updated
10	progress_percentage	Float	Default 0.0	Progress percentage of the task
11	assigned_to	Int	Foreign Key	Id of freelancer to whom task is assigned

17. tbl_freelancecontract

Primary key: id

Foreign key: client references table **tbl_customuser**, freelancer references table **tbl_customuser**, project references table **tbl_project**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	client	Int	Foreign Key	Id of client who associated with the project
3	freelancer	Int	Foreign Key	Id of freelancer who associated with the project
4	project	Int	One-to-One	Id of project the contract belongs to
5	client_signature	Varchar(file)	Null	File for client signature
6	freelancer_signature	Varchar(file)	Null	File for freelancer signature
7	contract_date	Date	Default Now	Date when the contract was signed
8	pdf_version	Varchar(file)	Null	PDF version of the contract

18. tbl_paymentinstallment

Primary key: id

Foreign key: contract references table **tbl_freelancecontract**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
-----	-----------	-----------------	-----------------	--------------------------

1	id	Int (Auto)	Primary Key	Primary key of the table
2	contract	Int	Foreign Key	Id of contract where the payments of a project specified
3	amount	Decimal(10, 2)	Not Null	Amount for the payment installment
4	due_date	Date	Not Null	Due date of the payment installment
5	status	Varchar(20)	Default 'pending'	Status of the installment (pending/paid)
6	razorpay_order_id	Varchar(255)	Null	Razorpay order ID for the installment
7	razorpay_payment_id	Varchar(255)	Null	Razorpay payment ID for the installment
8	paid_at	Date	Null	Date when the installment was paid

19. tbl_review

Primary key: id

Foreign key: project references table **tbl_project**, **reviewer** references table **tbl_customuser**, **reviewee** references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	project	Int	Foreign Key	Id of project which the client and freelancer collaborated
3	reviewer	Int	Foreign Key	Id of Reviewer
4	reviewee	Int	Foreign Key	Id of Reviewee
5	review_text	Text	Not Null	Text content of the review
6	overall_rating	Float	Not Null	Overall rating given in the review
7	quality_of_work	Float	Null	Rating for quality of work
8	communication	Float	Null	Rating for communication
9	adherence_to_deadlines	Float	Null	Rating for adherence to deadlines
10	professionalism	Float	Null	Rating for professionalism
11	problem_solving_ability	Float	Null	Rating for problem-solving ability

12	review_date	DateTime	Default Now	Timestamp when the review was made
----	-------------	----------	-------------	------------------------------------

20. tbl_chatroom**Primary key: id****Foreign key: project** references table **tbl_project**, **participants** references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	participants	Many-to-Many	Foreign Key	Id of participants
3	project	Int	Foreign Key	Id of project for which chatroom is creating
4	name	Varchar (255)	Null	Chatroom name (optional)
5	chat_type	Varchar (10)	Not Null, Default 'private'	Type of chat (Group/Private)

21. tbl_message**Primary key: id****Foreign key: chat_room** references table **tbl_chatroom**, **sender** references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	chat_room	Int	Foreign Key	Id of chatroom
3	sender	Int	Foreign Key	Id of user who send message
4	content	Text	Null	Text content of the message
5	image	Varchar (file)	Null	Image file in the message
6	file	Varchar (file)	Null	File in the message
7	timestamp	DateTime	Default Now	Timestamp of when the message was sent

22. tbl_complaint**Primary key: id****Foreign key: complaine** references table **tbl_customuser**, **user** references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	user	Int	Foreign Key	Id of Complainant
3	complaine	Int	Foreign Key	Id of Complaine
4	complaint_type	Varchar(20)	NOT NULL	Type of complaint (Client/Freelancer/Site Issue)
5	subject	Varchar(100)	NOT NULL	Subject of the complaint
6	description	Text	NOT NULL	Description of the complaint
7	status	Varchar(10)	Default 'Pending'	Status of the complaint (Pending/Resolved/Rejected)
8	created_at	DateTime	Default Now	Time of when the complaint was created
9	resolved_at	DateTime	Null	Time of when the complaint was resolved

23. tbl_template

Primary key: id

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	name	Varchar(255)		The name of the template
3	file	Varchar(255)		Path to the template file
4	cover_image	Varchar(255)	Null	Path to the cover image of the template

24. tbl_freelancerprofile

Primary key: id

Foreign key: user_id references table tbl_customuser

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	user_id	Int	Foreign Key	Login Id of freelancer
3	professional_title	Text(255)	Not Null	Freelancer's professional title

4	skills	Text(255)	Not Null	Freelancer's skills
5	experience_level	Varchar(50)	Null	Freelancer's experience level
6	portfolio_link	Varchar(255)	Null	Link to the freelancer's portfolio
7	education	Text	Null	Freelancer's education details
8	resume	Varchar(255)	Null	Path to the resume file
9	aadhaar_document	Varchar(255)	Null	Path to the Aadhaar document
10	status	Varchar(20)	Default 'active'	Current status of the freelancer profile (active/inactive)
11	work_type	Varchar(10)	Default 'part_time'	Work type (Full-time/Part-time)
12	is_open_to_work	Boolean	Default: False	Whether the freelancer is open to work
13	hiring_status	Varchar (20)	Default: 'NOT_HIRED'	Hiring status ('Hired' or 'Not Hired')

25. tbl_todo**Primary key: id****Foreign key: user_id** references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	user_id	Int	Foreign Key	Id of freelancer
3	title	Varchar(50)	Default None	Title of the to-do item
4	is_completed	Boolean	Default False	Status of the to-do item (Completed or not)
5	created_at	DateTime	Auto Add	Timestamp when the to-do item was created

26. tbl_proposal**Primary key: id****Foreign key: project_id** references table **tbl_project**, **freelancer_id** references table **tbl_customuser**, **team_id** references table **tbl_team**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table

2	project_id	Int	Foreign Key	Id of project where the proposal is associated
3	freelancer_id	Int	Foreign Key	Id of freelancer who sent proposal
4	date_issued	Date	Default Now	Date the proposal was issued
5	proposal_details	Text (Rich)		Details of the proposal
6	budget	Decimal(10, 2)		Proposed budget
7	deadline	Date		Proposal deadline
8	status	Varchar(10)	Default 'Pending'	Status of the proposal (Pending/Accepted/Rejected)
9	fancy_num	Varchar(5)	Unique, Null	Unique number associated with the proposal
10	proposal_file	Varchar(255)	Null	Path to the proposal file
11	locked	Boolean	Default False	Whether the proposal is locked or not
12	Team_id	Int	Foreign Key	Team id when submitted as team

27. tbl_proposalfile**Primary key: id****Foreign key: proposal_id** references table **tbl_proposal**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	proposal_id	Int	Foreign Key	Id of proposal submitted
3	file	Varchar(255)		Path to the uploaded file
4	uploaded_at	DateTime	Auto Add	Timestamp when the file was uploaded

28. tbl_document**Primary key: id****Foreign key: template_id** references table **tbl_template**, **user_id** references table **tbl_customuser**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	user_id	Int	Foreign Key	Foreign key Id of freelancer who owns the document

3	resume_file	Varchar(255)	Null	Path to the resume file
4	portfolio_file	Varchar(255)	Null	Path to the portfolio file
5	template_id	Int	Foreign Key	Id of template use choosed
6	created_at	DateTime	Auto Add	Timestamp when document created
7	cover_image	Varchar(255)	Null	Path to the cover image

29. tbl_ ChatbotKnowledge

Primary Key: id

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	query	Varchar (255)	Unique, Not Null	User's query input
3	response	Text	Not Null	Chatbot's response
4	created_at	DateTime	Not Null, Auto Now Add	Timestamp when record was created
5	updated_at	DateTime	Not Null, Auto Now	Timestamp when record was last updated

30. tbl_meeting

Primary Key: id

Foreign Key: project_id references table **tbl_project(id)**, **created_by_id** references table **tbl_customuser(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	title	Varchar (255)	Not Null	Title of the meeting
3	datetime	DateTime	Not Null	Date and time of the meeting
4	meeting_link	URL	Not Null	URL link for the meeting
5	project_id	Int	Foreign Key, Not Null	Associated project
6	created_by	Int	Foreign Key, Not Null	User who created the meeting
7	attendees	ManyToMany	Not Null	Users attending the meeting
8	status	Varchar (20)	Default: 'scheduled'	Status of the meeting

9	created_at	DateTime	Auto Now Add, Not Null	Meeting creation timestamp
10	updated_at	DateTime	Auto Now, Not Null	Meeting last update timestamp

31. tbl_job_invitation**Primary Key: id****Foreign Key:freelancer_id** references table **tbl_customuser(id)**,**client_id** references table **tbl_customuser(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	freelancer_id	Int	Foreign Key, Not Null	ID of the freelancer receiving the invitation
3	client_id	Int	Foreign Key, Null	ID of the client sending the invitation
4	job_role	Varchar (255)	Not Null	Job role offered
5	job_description	Text	Not Null	Description of the job
6	compensation	Varchar (100)	Not Null	Compensation details
7	created_at	DateTime	Auto Timestamp	Time when invitation was created
8	status	Varchar (10)	Default: 'Pending'	Status of invitation (Pending, Accepted, Rejected)
9	meeting_link	URL	Null	Meeting link for discussion
10	meeting_datetime	DateTime	Null	Scheduled date and time for the meeting
11	work_mode	Varchar (50)	Default: 'Specify Later'	Work mode (Remote, Hybrid, On-Site)
12	benefits	Text	Null	Benefits provided
13	bond_period	Varchar (100)	Null	Minimum bond period
14	contract_needed	Boolean	Default: False	Whether a contract is required
15	hiring_status	Varchar (10)	Default: 'Pending'	Hiring status (Hired, Not Hired, Pending)

32. tbl_event_and_quiz**Primary Key: id****Foreign Key: client_id** references table **tbl_customuser(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	title	Varchar (255)	Not Null	Title of the event/quiz
3	description	Text	Not Null	Event or quiz description
4	date	DateTime	Not Null	Event or quiz date
5	type	Varchar (10)	Not Null	Type (Event, Quiz)
6	type_of_event	Varchar (20)	Null	Type of event (Webinar, Workshop, etc.)
7	poster	File (255)	Null	Path to event poster
8	max_participants	Int	Null	Maximum allowed participants
9	online_link	URL	Null	Link to join online events
10	certificate_provided	Boolean	Default: False	Whether a certificate is provided
11	prize_enabled	Boolean	Default: False	Whether prize is enabled
12	prize_amount	Decimal (10,2)	Null	Prize amount
13	client_id	Int	Foreign Key, Null	ID of client who created the event
14	quiz_file	File (255)	Null	File containing quiz details
15	duration	Duration	Null	Quiz duration
16	registration_status	Varchar (10)	Default: 'open'	Registration status (Open, Closed)
17	registration_end_date	DateTime	Null	Registration deadline
18	number_of_registrations	Int	Default: 0	Number of registrations
19	event_status	Varchar (20)	Default: 'Upcoming'	Event status (Upcoming, Done)
20	certificate_template	File (255)	Null	Path to certificate template
21	approval_status	Varchar (20)	Default: 'Pending'	Approval status of event/quiz
22	rejection_reason	Text	Null	Reason for rejection
23	created_at	DateTime	Auto Timestamp	Event creation timestamp
24	updated_at	DateTime	Auto Timestamp	Last update timestamp

33. tbl_quiz_question

Primary Key: id

Foreign Key: quiz_id references table **tbl_event_and_quiz(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	quiz_id	Int	Foreign Key, Not Null	ID of the related quiz
3	question	Text	Not Null	Quiz question content
4	option1	Varchar (255)	Not Null	First option
5	option2	Varchar (255)	Not Null	Second option
6	option3	Varchar (255)	Not Null	Third option
7	option4	Varchar (255)	Not Null	Fourth option
8	correct_answer	Varchar (255)	Not Null	Correct answer
9	points	Int	Not Null	Points awarded for correct answer

34. tbl_event_registration

Primary Key: id

Foreign Key:freelancer_id references table **tbl_customuser(id)**,**event_id** references table **tbl_event_and_quiz(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	freelancer_id	Int	Foreign Key, Not Null	Freelancer registering for the event
3	event_id	Int	Foreign Key, Not Null	Related event
4	registration_time	DateTime	Auto Timestamp	Registration timestamp
5	status	Varchar (50)	Default: 'registered'	Registration status
6	seen_popup	Boolean	Default: False	Whether the freelancer has seen the popup
7	attended	Boolean	Default: False	Whether the freelancer attended

35. tbl_prize_payment

Primary Key: id

Foreign Keys: event_id references table **tbl_event_and_quiz(id)**, winner_id references table **tbl_customuser(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	event_id	Int	Foreign Key, Not Null	Related event
3	winner_id	Int	Foreign Key, Not Null	ID of the winner
4	amount	Decimal (10,2)	Not Null	Prize amount
5	payment_date	DateTime	Auto Timestamp	Payment date
6	payment_status	Varchar (20)	Default: 'Pending'	Payment status
7	razorpay_order_id	Varchar (100)	Null	Razorpay order ID
8	razorpay_payment_id	Varchar (100)	Null	Razorpay payment ID
9	razorpay_signature	Varchar (255)	Null	Razorpay signature

36. tbl_team

Primary Key: id

Foreign Key: created_by references table **tbl_customuser(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	name	Varchar (100)	Not Null	Name of the team
3	created_by	Int	Foreign Key, Not Null	User who created the team
4	created_at	DateTime	Auto Now Add	Timestamp when the team was created
5	join_code	Varchar (8)	Unique	Unique join code for the team

37. tbl_team_member

Primary Key: id

Foreign Key: team_id references table **tbl_team(id)**, user_id references table **tbl_customuser(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table

2	team_id	Int	Foreign Key, Not Null	Team the user belongs to
3	user_id	Int	Foreign Key, Null	ser who is a team member
4	role	Varchar (20)	Not Null	Role of the member in the team
5	joined_at	DateTime	Auto Now Add	Timestamp when the user joined the team
6	is_active	Boolean	Default: True	Whether the user is active in the team
7	salary_percentage	Decimal (5,2)	Null	Salary share percentage of the member

38. tbl_team_invitation**Primary Key: id****Foreign Key:team_id** references table **tbl_team(id)**,**invited_by** references table **tbl_customuser(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	team_id	Int	Foreign Key, Not Null	Team being invited to
3	email	Varchar (255)	Not Null	mail of the invited person
4	role	Varchar (20)	Not Null	Role for which user is invited
5	invited_by	Int	Foreign Key, Not Null	User who sent the invitation
6	created_at	DateTime	Auto Now Add	Timestamp when invitation was created
7	token	Varchar (100)	Unique, Not Null	Unique token for invitation
8	status	Varchar (20)	Default: 'pending'	Invitation status (Pending, Accepted, Rejected)
9	expires_at	DateTime	Default: now() + 15 days	Expiration date of the invitation

39. tbl_salary_payment**Primary Key: id****Foreign Key:team_member_id** references table **tbl_team_member(id)**,**project_id** references table **tbl_project(id)**,**paid_by** references table **tbl_customuser(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Primary key of the table
2	team_member_id	Int	Foreign Key, Not Null	Team member receiving the payment
3	project_id	Int	Foreign Key, Not Null	Project for which payment is made
4	amount_paid	Decimal (10,2)	Not Null	Amount paid to the team member
5	payment_date	DateTime	Auto Now Add	Timestamp when payment was made
6	paid_by	Int	Foreign Key, Null	User who made the payment
7	status	Varchar (20)	Default: 'pending'	Payment status (Pending, Completed, Failed)

40. tbl_subscription_plan

Primary Key: id

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Unique identifier for each plan
2	name	Varchar (100)	Unique, Not Null	Name of the plan (Basic, Premium, Pro)
3	price	Decimal (10,2)	Not Null	Price of the plan
4	duration_days	Int	Not Null	Duration of the plan in days
5	is_active	Boolean	Default: True	Whether the plan is currently active
6	can_hire_freelancer	Boolean	Default: False	Allow clients to hire freelancers
7	can_create_events	Boolean	Default: False	Allow clients to create events
8	project_creation_limit	Int	Not Null, Default: 0	Maximum projects a client can create
9	ai_freelancer_recommendations	Boolean	Default: False	Enable AI freelancer recommendations
10	can_participate_events	Boolean	Default: False	Allow freelancers to participate in events
11	ai_skill_gap_analysis	Boolean	Default: False	Enable AI skill gap analysis
12	can_create_portfolio	Boolean	Default: False	Allow freelancers to create portfolios
13	max_proposals_limit	Int	Not Null, Default: 0	Maximum proposals a freelancer can submit

14	ai_project_recom mendations	Boolean	Default: False	Enable AI project recommendations
15	team_creation	Boolean	Default: False	Allow freelancers to create teams
16	created_at	DateTime	Auto Now Add	Timestamp when the plan was created
17	updated_at	DateTime	Auto Now	Timestamp when the plan was last updated

41. tbl_user_subscription**Primary Key: id****Foreign Key:** user_id references table **tbl_customuser(id)**, plan_id references table **tbl_subscription_plan(id)**

No:	Fieldname	Datatype (Size)	Key Constraints	Description of the Field
1	id	Int (Auto)	Primary Key	Unique identifier for each subscription
2	user_id	Int	Foreign Key, Not Null	User who purchased the subscription
3	plan_id	Int	Foreign Key, Null	Subscription plan selected
4	amount	Decimal (10,2)	Null	Amount paid for the plan
5	payment_status	Varchar (20)	Default: 'Pending'	Payment status (Pending, Completed, Failed)
6	razorpay_order_i d	Varchar (255)	Null	Order ID from Razorpay
7	razorpay_paymen t_id	Varchar (255)	Null	Payment ID from Razorpay
8	start_date	DateTime	Auto Now Add	Subscription start date
9	end_date	DateTime	Null	Subscription expiration date
10	is_active	Boolean	Default: True	Whether the subscription is active
11	created_at	DateTime	Auto Now Add	Timestamp when subscription was created
12	updated_at	DateTime	Auto Now	Timestamp when subscription was last updated

CHAPTER 5

SYSTEM TESTING

5.1 INTRODUCTION

Testing identifies bugs and deviations in the software early and making sure that software is of the highest quality, reliability and functionality. Unit, integration, system, acceptance and regression testing are the key types. It enhances the security, performance and quality standards compliance. Testing minimizes risks and lowers costs by identifying problems before deployment and helps improve the customer satisfaction which results into a robust and reliable product.

5.2 TEST PLAN

A test plan is a formal document which defines the strategy of testing an application. It specifies test environment, test cases, tools, data and team roles in order to guide the testing team. So, the main goal is the process of testing structured and efficiently. In addition, it contains informative details on the type of testing to be done at various levels, helping you scan, locate and eliminate problems systematically. These typically include:

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

5.2.1 Unit Testing

Software testing method that tests code components or function in isolation. It guarantees that each unit works as expected, according to its requirements. Automated test cases are a tool used by developers to catch issues early in the development process and thereby code easier to maintain, reliable and easier to debug.

5.2.2 Integration Testing

Integration testing tests how various parts of a system combine together in regard to the flow of data between modules and how they interact with each other. Built on top of the unit testing, it helps to find issues that aren't caught by unit tests, guarantees a clean integration of subsystems and external interfaces. Top down, bottom up or even hybrid methods are the common approaches. The intention is to verify that when combined parts are used together that the system behaves properly before it leaves the lab.

5.2.3 Validation Testing or System Testing

The validation testing is intended to test the software to confirm if it works as per stakeholder's and user's needs. While system testing assesses all of the entire system's functionality, reliability, and interaction amongst components. Together they are a key part on making sure the software is

complete, functional, and ready to be released.

5.2.4 Output Testing or User Acceptance Testing

It is the final phase in the testing and the statements will be made by the end users to ensure that software meets the business needs and is performing as expected. It guarantees that the system is operation-friendly, and reliable with functional as well as non-functional requirements. That is to say UAT of the software confirms that it is ready for deployment.

5.2.5 Automation Testing

Automation testing is done using tools, scripts to carry out automatically test cases so it is faster, accurate and consistent than the manual testing. This is useful for functional and regression testing, especially it gives early detection ability to identify problems, reduction of costs, and concomitantly improves overall software quality.

5.2.6 Selenium Testing

Selenium is an open-source framework used to automate web browsers and used for functional and regression testing of web applications. It is able to support multiple languages of programming including Java, C#, and Python, and it can simulate user interactions like typing and clicking on various browsers such as Chrome and Firefox. With features of creating test scripts, running and reporting Selenium is a well-integrated with an open source existing TestNG, Junit, Jenkins and others to aptly suit the environment of the software and its industry.

Example:

Test Case 1

Code

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
login_data = [{"email": "admin@gmail.com", "password": "Admin@12", "expected_url":
    "http://127.0.0.1:8000/administrator/admin_view/"}, {"email": "ava.robinson@gmail.com",
    "password": "Ava@1234", "expected_url":
    "http://127.0.0.1:8000/freelancer/freelancer_view/"}, {"email": "samuel@gmail.com",
    "password": "Sam@1234", "expected_url":
```

```

"http://127.0.0.1:8000/client/client_view/"},"email": "samuel@gmail.com", "password":
"Sam@1235", "expected_url": "http://127.0.0.1:8000/login/"}], ]
try:
    for credentials in login_data:
        driver.get("http://127.0.0.1:8000/login/")
        email_input = WebDriverWait(driver, 10).until(
            EC.visibility_of_element_located((By.NAME, "mail")))
        password_input = WebDriverWait(driver, 10).until(
            EC.visibility_of_element_located((By.NAME, "pass")))
        email_input.clear()
        password_input.clear()
        email_input.send_keys(credentials['email'])
        password_input.send_keys(credentials['password'])
        password_input.send_keys(Keys.RETURN)
        time.sleep(2)
        if driver.current_url == credentials['expected_url'] and
        driver.current_url!="http://127.0.0.1:8000/login/":
            print(f"Login successful for {credentials['email']}")
            time.sleep(2)
            logout_button = WebDriverWait(driver, 10).until(
                EC.element_to_be_clickable((By.LINK_TEXT, "Logout")))
            logout_button.click()
        else:
            print(f"Login failed. Incorrect URL after login.")
    finally:
        driver.quit()

```

Screenshot

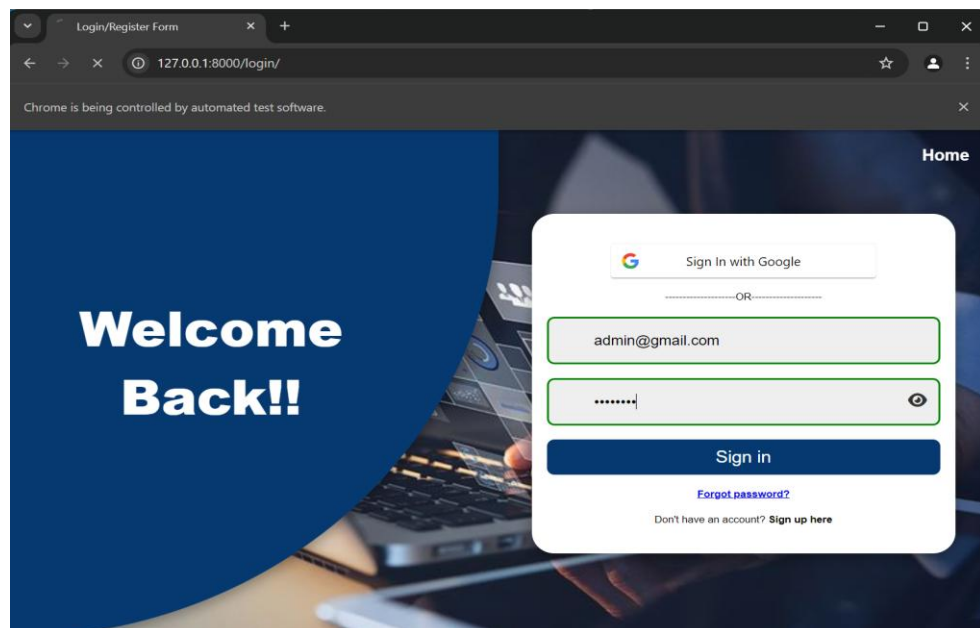


Figure 18: Login page with valid credentials entered

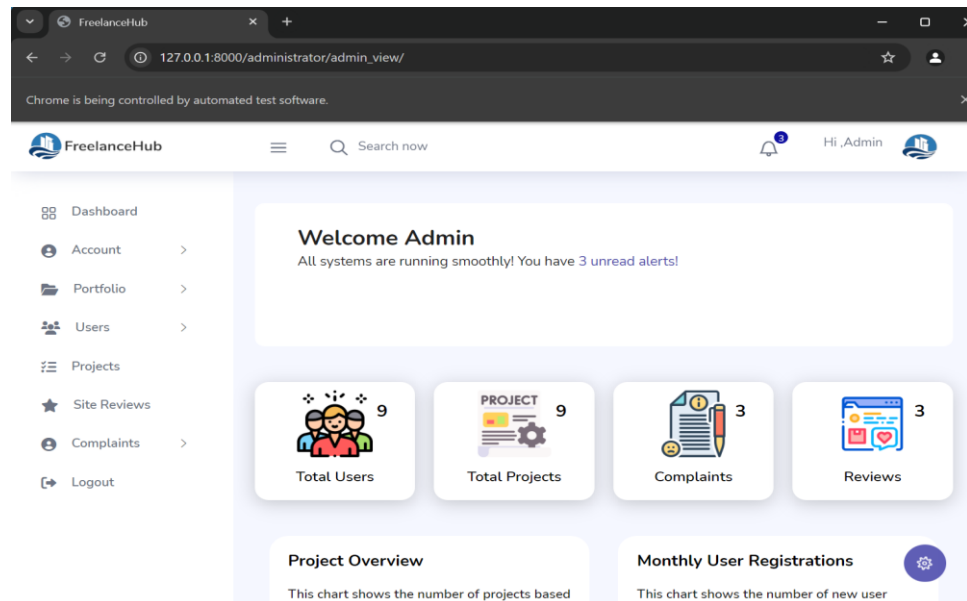


Figure 19:Admin dashboard after successful Login page

```
PS C:\Users\LENOVO\Desktop\Freelancehub> & "C:/Program Files/python/python.exe" c:/Users/LENOVO/Desktop/freelancehub/tests/test_login.py
DevTools listening on ws://127.0.0.1:58105/devtools/browser/5b40227f-9736-4eae-8047-e6d30eb8d806
Login successful for admin@gmail.com.
Created TensorFlow Lite XNNPACK delegate for CPU.
Attempting to use a delegate that only supports static-sized tensors with a graph that has dynamic-sized tensors (tensor#58 is a dynamic-sized tensor).
Login successful for ava.robinson@gmail.com.
Login successful for samuel@gmail.com.
Login failed. Incorrect URL after login.
```

Figure 20: Login page Test result

Test Report

Test Case 1	
Project Name:FreelanceHub	
Login Test Case	
Test Case ID: Test_1	Test Designed By: Varsha Shaji
Test Priority(Low/Medium/High):High	Test Designed Date: 09/03/2025
Module Name: Login Page	Test Executed By: Ms. Meera Rose Mathew
Test Title : Verify Login with valid email and password	Test Execution Date: 10/03/2025
Description: Login page Testing	
Pre-Condition: User has valid email and password	

Step	Test Step	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Navigate to login page		Display Login page	Login page displayed	Pass
2	Valid email entered	Email: admin@gmail.com	User should be able to login	User logged in and navigate to the admin dashboard page.	Pass
3	Valid password entered	Password: Admin@12			
4	Click on Login Button				
Post-Condition: Email id and password is checked with database values for successful Login					

Test Case 2:**Code**

```

from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
registration_data = [
    {"fname": "Mary", "lname": "Doe", "email": "mary.doe@gmail.com", "password":
    "Mary@123",
    "repassword": "Mary@123", "expected_url": "http://127.0.0.1:8000/add_user_type/65"
    },
    {"fname": "Noah", "lname": "Wilson", "email": "noah.wilson@gmail.com", "password":
    "Noah@123", "repassword": "Noah@123", "expected_url": "http://127.0.0.1:8000/register/"}
]

try:
    for data in registration_data:
        driver.get("http://127.0.0.1:8000/register/")

```

```
fname_input = WebDriverWait(driver, 10).until(
    EC.visibility_of_element_located((By.NAME, "fname")))
lname_input = WebDriverWait(driver, 10).until(
    EC.visibility_of_element_located((By.NAME, "lname")))
email_input = WebDriverWait(driver, 10).until(
    EC.visibility_of_element_located((By.NAME, "email")))
password_input = WebDriverWait(driver, 10).until(
    EC.visibility_of_element_located((By.NAME, "password")))
repassword_input = WebDriverWait(driver, 10).until(
    EC.visibility_of_element_located((By.NAME, "repassword")))
fname_input.clear()
lname_input.clear()
email_input.clear()
password_input.clear()
repassword_input.clear()
fname_input.send_keys(data['fname'])
lname_input.send_keys(data['lname'])
email_input.send_keys(data['email'])
password_input.send_keys(data['password'])
repassword_input.send_keys(data['repassword'])
driver.find_element(By.ID, "subbtn").click()
time.sleep(3)
if driver.current_url == data['expected_url'] and "add_user_type" in data['expected_url']:
    driver.find_element(By.ID, "create-account-btn").click()
    time.sleep(3)
    if driver.current_url == "http://127.0.0.1:8000/login/":
        print(f"Test Passed :Registration successful for {data['email']}")
    else:
        print(f"Test failed for {data['email']}")
elif driver.current_url == data['expected_url'] and data['expected_url'] ==
"http://127.0.0.1:8000/register/":
    print(f"Test passed for existing email {data['email']}")
else:
    print(f"Test failed for {data['email']}")
```

finally:

```
driver.quit()
```

Screenshot

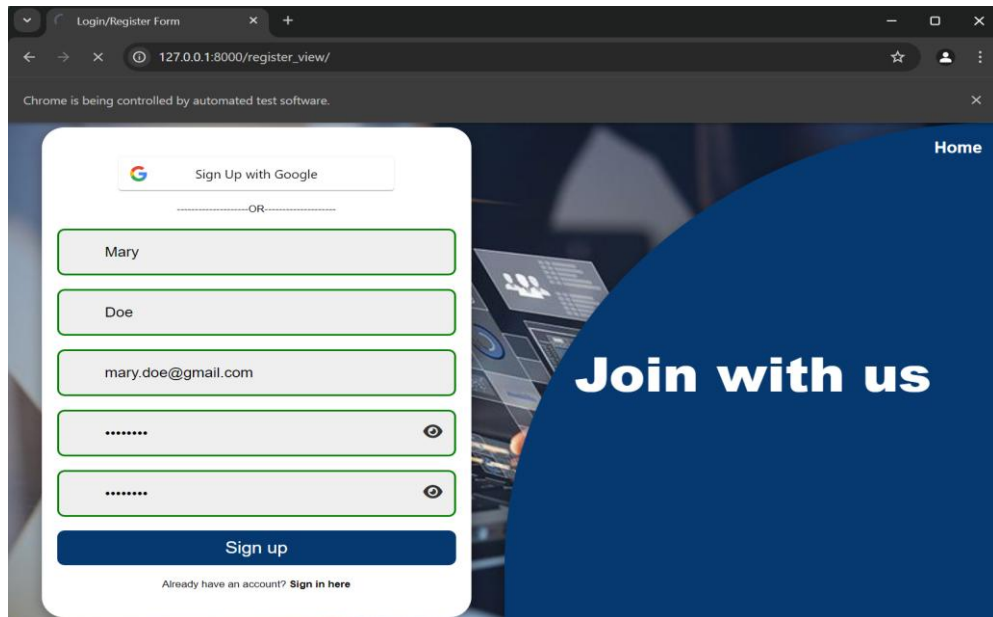


Figure 21: Register page with valid credentials entered

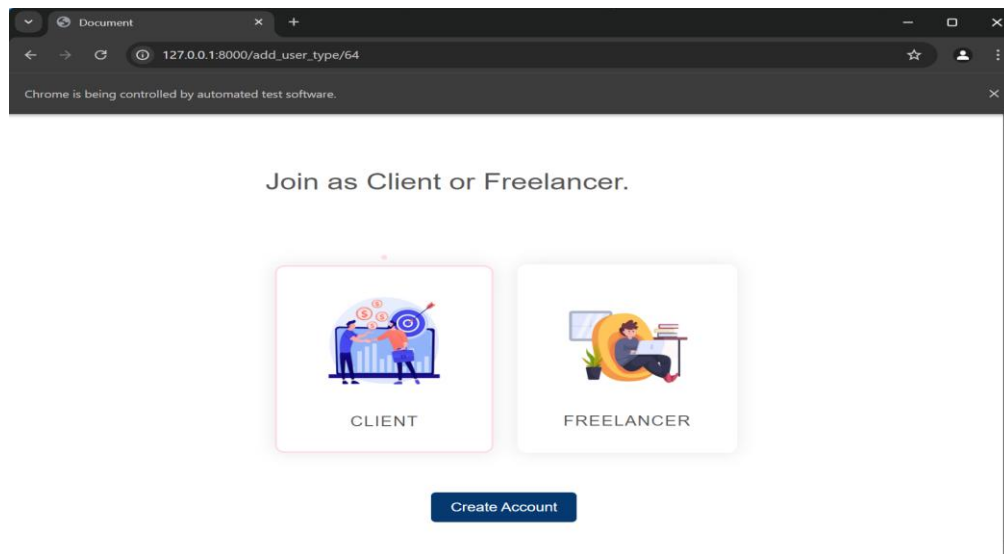


Figure 22: Role choose page after successful registration

```
PS C:\Users\LENOVO\Desktop\freelancehub> & "C:/Program Files/python/python.exe" c:/Users/LENOVO/Desktop/freelancehub/tests/test_register.py

DevTools listening on ws://127.0.0.1:58097/devtools/browser/0250af10-83d5-4bb3-8ff6-26a6c817f703
Test Passed :Registration successful for mary.doe@gmail.com
Created TensorFlow Lite XNNPACK delegate for CPU.
Attempting to use a delegate that only supports static-sized tensors with a graph that has dynamic-sized tensors (tensor#58 is a dynamic-sized tensor).
Test passed for existing email noah.wilson@gmail.com
```

Figure 23: Register page Test result

Test report

Test Case 2					
Project Name: FreelanceHub					
Register Test Case					
Test Case ID: Test_2			Test Designed By: Varsha Shaji		
Test Priority(Low/Medium/High):High			Test Designed Date: 09/03/2025		
Module Name: Registration Page			Test Executed By: Ms. Meera Rose Mathew		
Test Title : Verify registration with valid credentials			Test Execution Date: 10/03/2025		
Description: Registration page Testing					
Pre-Condition :User has valid credentials					
Step	Test Step	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Navigate to register page		Display registration page	Registration page displayed	Pass
2	Valid details are entered	fname:Mary lname:Doe email: mary.doe@gmail.com password:Mary@123 confirm password: Mary@123	User should be able to register	User navigate to the user type selecting page.	Pass
3	Click on register Button				
4	Navigate to add user type page	Display add user type page	Add user type page displayed	Add user type Page displayed	Pass
5	Select user type	user_type:client	User should be able to add user type	User navigate to the user type selecting page.	Pass
6	Click on create account Button				
Post-Condition: Details are validated with database values for successful registration					

Test Case 3:**Code**

```

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys

```

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
import time
options = webdriver.ChromeOptions()
options.add_argument("--disable-gpu")
driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=options)
try:
    driver.get('http://127.0.0.1:8000/login/')
    email_input = WebDriverWait(driver, 10).until(EC.element_to_be_clickable((By.NAME,
'mail'))))
    email_input.send_keys('creativdesign@gmail.com')

    password_input = driver.find_element(By.NAME, 'pass')
    password_input.send_keys('Creative@1')
    password_input.send_keys(Keys.RETURN)

    WebDriverWait(driver, 10).until(EC.url_contains('/client/client_view/'))
    driver.get('http://127.0.0.1:8000/client/track_hiring/')
    schedule_button = WebDriverWait(driver,
10).until(EC.element_to_be_clickable((By.CSS_SELECTOR, '.schedule-btn:not([disabled])'))))
    schedule_button.click()
    modal = WebDriverWait(driver, 10).until(EC.visibility_of_element_located((By.ID,
'scheduleModal'))))
    meeting_link = driver.find_element(By.ID, 'meetingLink')
    meeting_link.clear()
    meeting_link.send_keys('https://google.com/abc-defg-hij')
    meeting_date = driver.find_element(By.ID, 'meetingDate')
    meeting_date.clear()
    meeting_date.send_keys('02/04/2025 10:00')
    submit_button = driver.find_element(By.ID, 'submitBtn')
    is_disabled = submit_button.get_attribute('disabled')

    if is_disabled is not None:
        print("✅ Test Passed: Submit button is disabled with an invalid meeting link.")
    else:
        print("❌ Test Failed: Submit button is enabled despite an invalid meeting link.")

except Exception as e:
    print(f"❌ Test Failed: {str(e)}")

finally:
    driver.quit()
```

Screenshot

Figure 24: Meeting schedule form with invalid data

```

PS C:\Users\LENOVO\OneDrive\Desktop\freelancehub> & "C:/Program Files/python/python.exe" c:/Users/LENOVO/OneDrive/Desktop/freelancehub/tests/test_meeting_scheduling.py

DevTools listening on ws://127.0.0.1:53354/devtools/browser/043994b5-f7c7-40db-9f74-bafc45d526d8
✓ Test Passed: Submit button is disabled with an invalid meeting link.

```

Figure 25: Meeting schedule form Test result

Test Report

Test Case 3	
Project Name: FreelanceHub	
Meeting Schedule Creation Test Case	
Test Case ID: Test_3	Test Designed By: Varsha Shaji
Test Priority(Low/Medium/High):High	Test Designed Date: 09/03/2025
Module Name: Meeting Schedule creation Page	Test Executed By: Ms.Meera Rose Mathew
Test Title: Verify Creating meeting schedule with invalid credentials	Test Execution Date: 10/03/2025
Description: Portfolio Template Adding Testing	
Pre-Condition: Providing Invalid credentials	

Step	Test Step	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Navigate to login page		Display login page	Login page displayed	Pass
2	Valid details are entered	email: creativedesign@gmail.com password: Creative@1	User should be able to login	User navigate to the client dashboard	Pass
3	Click on login Button				
4	Navigate to track hiring page	Display hiring tracking page	Display hiring tracking	hiring tracking displayed	Pass
5	Datas entered	link:https://google.com/abc-defg-hij date:02/04/2025	User shouldn't be able to create schedule	User couldn't create meeting schedule.	Pass
6	Click on submit Button				
Post-Condition: client cannot create meeting schedule because of invalid credentials					

Test Case 4:**Code**

```

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
import time
GREEN = '\033[92m'
RED = '\033[91m'
RESET = '\033[0m'

driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
try:
    driver.get('http://127.0.0.1:8000/login/')
    email_input = WebDriverWait(driver, 10).until(EC.element_to_be_clickable((By.NAME, 'mail')))
    email_input.send_keys('noah.wilson@gmail.com')
    password_input = WebDriverWait(driver, 10).until(EC.element_to_be_clickable((By.NAME, 'pass')))
    password_input.send_keys('Noah@123')
    password_input.send_keys(Keys.RETURN)

```



```
try:
    WebDriverWait(driver, 10).until(EC.url_contains('/freelancer/freelancer_view/'))
    print(f'{GREEN}Logged in successfully.{RESET}')
except Exception as e:
    print(f'{RED>Login failed: {str(e)}{RESET}')
    raise
driver.get('http://127.0.0.1:8000/freelancer/manage-team/')
try:
    create_team_btn = WebDriverWait(driver, 10).until(EC.element_to_be_clickable((By.ID,
    'createTeamBtn'))))
    create_team_btn.click()
except Exception as e:
    print(f'{RED}Failed to find or click create team button: {str(e)}{RESET}')
    raise
try:
    team_name_input = WebDriverWait(driver, 10).until(
        EC.element_to_be_clickable((By.ID, 'teamNameInput'))
    )
    team_name_input.send_keys('hi')

    time.sleep(5)
    WebDriverWait(driver, 10).until(
        lambda x: x.find_element(By.ID, 'submitButton').get_attribute('disabled') == 'true'
    )
    close_button = driver.find_element(By.ID, 'closeButton')
    submit_button = driver.find_element(By.ID, 'submitButton')
    if submit_button.get_attribute('disabled'):
        print(f'{GREEN}Test Passed: Submit button is disabled for existing team
name.{RESET}')
        close_button.click()
        driver.refresh()
    else:
        print(f'{RED}Test Failed: Submit button should be disabled for existing team
name.{RESET}')
        time.sleep(5)
        create_team_btn = WebDriverWait(driver, 10).until(
            EC.element_to_be_clickable((By.ID, 'createTeamBtn'))
        )
        create_team_btn.click()

        team_name_input = WebDriverWait(driver, 10).until(
            EC.element_to_be_clickable((By.ID, 'teamNameInput'))
        )
        team_name_input.send_keys('New Test Team')

        WebDriverWait(driver, 10).until(
            lambda x: x.find_element(By.ID, 'submitButton').get_attribute('disabled') != 'true'
        )

        submit_button = driver.find_element(By.ID, 'submitButton')
        if not submit_button.get_attribute('disabled'):
```

```

submit_button.click()
WebDriverWait(driver, 10).until(EC.url_contains('/manage-team/'))

WebDriverWait(driver, 10).until(
    EC.presence_of_element_located((By.ID, 'createTeamBtn'))
)
print(f'{GREEN}Test Passed: Team created successfully with new name.{RESET}')
else:
    print(f'{RED}Test Failed: Submit button should be enabled for new team name.{RESET}')
except Exception as e:
    print(f'{RED}Failed during team creation tests: {str(e)}{RESET}')
    raise

except Exception as e:
    print(f'{RED}Test Failed with error: {str(e)}{RESET}')
    raise

finally:
    driver.quit()

```

Screenshot

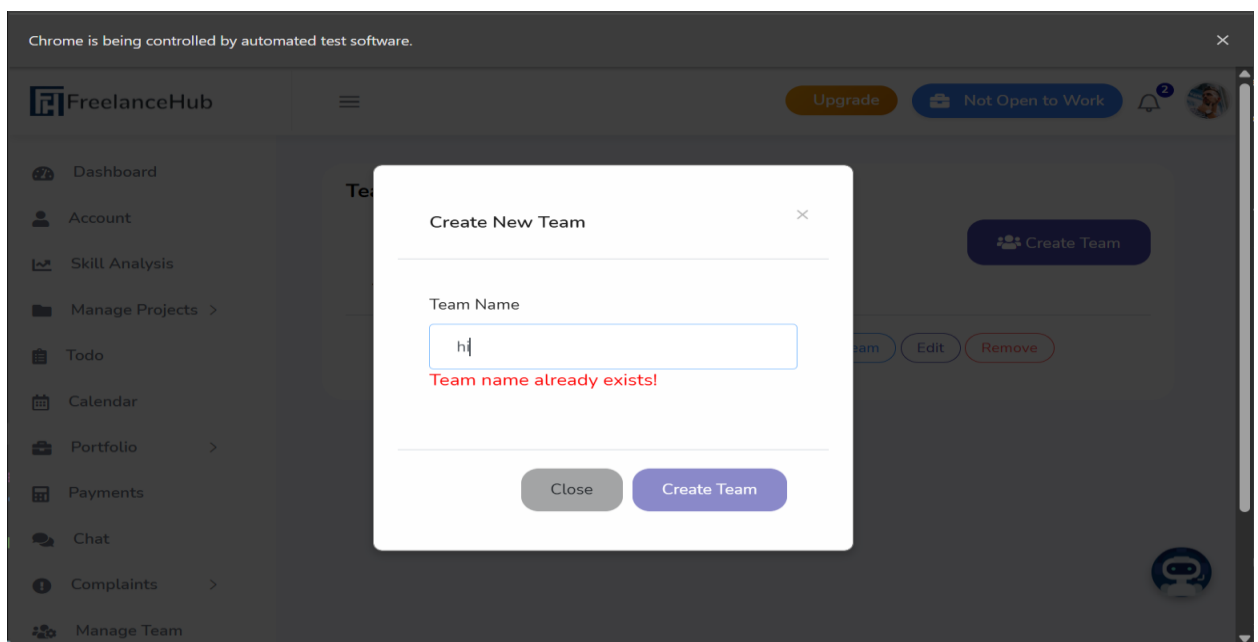


Figure 26: Team creation form

```

PS C:\Users\LENOVO\OneDrive\Desktop\freelancehub> & "C:/Program Files/python/python.exe" c:/Users/LENOVO/OneDrive/Desktop/freelancehub/tests/test_team_forms.py
fed-a343-5c413f425e03
Logged in successfully.
Test Passed: Submit button is disabled for existing team name.
Created TensorFlow Lite XNNPACK delegate for CPU.
Attempting to use a delegate that only supports static-sized tensors with a graph that has dynamic-sized tensors (tensor#-1 is a dynamic-sized tensor).
Test Passed: Team created successfully with new name.

```

Figure 27: Team creation form Test result

Test Report

Test Case 4					
Project Name: FreelanceHub					
New Team Creation Test Case					
Test Case ID: Test_4			Test Designed By: Varsha Shaji		
Test Priority(Low/Medium/High):High			Test Designed Date: 09/03/2025		
Module Name: New Team Creation Page			Test Executed By: Ms. Meera Rose Mathew		
Test Title: Verify Creating new team name with invalid credentials			Test Execution Date: 10/03/2025		
Description: New Team Creation form Testing					
Pre-Condition: Providing Invalid credentials					
Step	Test Step	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Navigate to login page		Display login page	Login page displayed	Pass
2	Valid details are entered	email: noah.wilson@gmail.com password: Noah@123	User should be able to login	User navigate to the freelancer's dashboard	Pass
3	Click on login Button				
4	Navigate to manage team page and click on create new team button.	Display manage team page and click on create new team button page	Display Form for team creation	Form for team creation displayed	Pass
5	Data entered	name: hi	User shouldn't be able to create team	Shows error in form and wouldn't able to create team.	Pass
6	Click on submit Button				
Post-Condition: Freelancer cannot create team because of invalid credentials					

Test Case 5:**Code**

```

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait

```

```
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
import time

GREEN = '\033[92m'
RED = '\033[91m'
RESET = '\033[0m'
driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
try:
    driver.get('http://127.0.0.1:8000/login/')
    email_input = WebDriverWait(driver, 10).until(
        EC.element_to_be_clickable((By.NAME, 'mail'))
    )
    email_input.send_keys('vshaji596@gmail.com')
    password_input = WebDriverWait(driver, 10).until(
        EC.element_to_be_clickable((By.NAME, 'pass'))
    )
    password_input.send_keys('Ava@1234')
    password_input.send_keys(Keys.RETURN)
    try:
        WebDriverWait(driver, 10).until(EC.url_contains('/freelancer/freelancer_view/'))
        except Exception as e:
            raise
    driver.get('http://127.0.0.1:8000/freelancer/analyze-skill-gap/')
    try:
        input_field = WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.ID, "jobRoleInput"))
        )
        analyze_button = driver.find_element(By.ID, "analyzeBtn")
        error_message = driver.find_element(By.ID, "jobRoleError")

        input_field.clear()
        input_field.send_keys("Front-end @Developer")
        time.sleep(1)

        if analyze_button.get_attribute('disabled') and error_message.is_displayed():
            print(f'{GREEN}Test Passed: Button disabled and error shown for invalid input 'Front-
            end @Developer'{RESET}")
        else:
            print(f'{RED}Test Failed: Button should be disabled for invalid input 'Front-end
            @Developer'{RESET}")
        except Exception as e:
```

```

    print(f"{RED}Failed during job role validation tests: {str(e)}{RESET}")
    raise
except Exception as e:
    print(f"{RED}Test Failed with error: {str(e)}{RESET}")
    raise
finally:
    driver.quit()

```

Screenshot

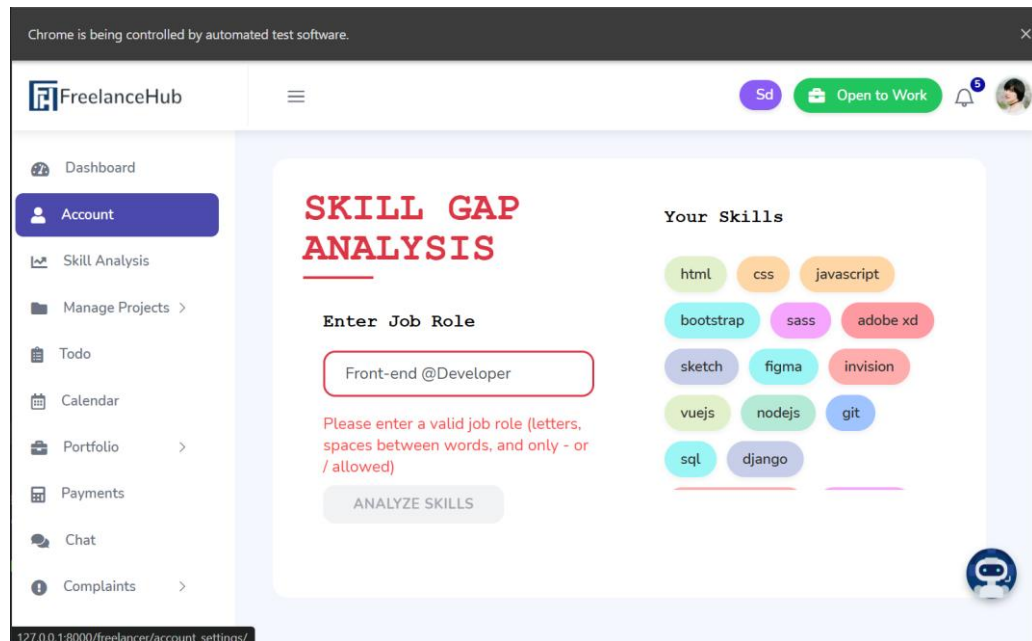


Figure 28: Skill gap analysis form with invalid data entered

```

PS C:\Users\LENOVO\OneDrive\Desktop\freelancehub> & "C:/Program Files/python/pyt
hon.exe" c:/Users/LENOVO/OneDrive/Desktop/freelancehub/tests/test_skill_gap.py

DevTools listening on ws://127.0.0.1:62982/devtools/browser/cb29a2ec-731a-40c1-9
29d-781235ad5aa2
Test Passed: Button disabled and error shown for invalid input 'Front-end @Devel
oper'

```

Figure 29: Skill gap analysis form Test result

Test Report

Test Case 5	
Project Name: FreelanceHub	
Skill Gap Analysis Test Case	
Test Case ID: Test_5	Test Designed By: Varsha Shaji
Test Priority(Low/Medium/High):High	Test Designed Date: 09/03/2025
Module Name: Skill Gap Analysis Form	Test Executed By: Ms. Meera Rose Mathew

Test Title: Verify job role in skill gap analysis with invalid credentials			Test Execution Date: 10/03/2025		
Description: Skill Gap Analysis form Testing					
Pre-Condition: Providing Invalid credentials					
Step	Test Step	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Navigate to login page		Display login page	Login page displayed	Pass
2	Valid details are entered	email: vshaji596@gmail.com password: Ava@1234	User should be able to login	User navigate to the freelancer’s dashboard	Pass
3	Click on login Button				
4	Navigate to skill gap analysis page		Display skill gap analysis page	skill gap analysis page displayed	Pass
5	Data entered	Job role: Front-end @Developer	User shouldn’t be able to analyze	Shows error in form and wouldn’t able to analyze.	Pass
6	Click on submit Button				
Post-Condition: Freelancer cannot analyze skill because of invalid credentials					

Test Case 6:**Code**

```

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
import time

```

```

GREEN = '\033[92m'
RED = '\033[91m'
BLUE = '\033[94m'
BOLD = '\033[1m'
END = '\033[0m'

```

```
def wait_for_element(driver, by, value, timeout=10):
    return WebDriverWait(driver, timeout).until(
        EC.element_to_be_clickable((by, value))
    )
def wait_for_element_visible(driver, by, value, timeout=10):
    return WebDriverWait(driver, timeout).until(
        EC.visibility_of_element_located((by, value))
    )
driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()))
try:
    driver.get('http://127.0.0.1:8000/login/')
    email_input = wait_for_element(driver, By.NAME, 'mail')
    email_input.send_keys('admin@gmail.com')
    password_input = wait_for_element(driver, By.NAME, 'pass')
    password_input.send_keys('Admin@12')
    password_input.send_keys(Keys.RETURN)
    WebDriverWait(driver, 10).until(EC.url_contains('/administrator/admin_view/'))
    print(f'{GREEN}{BOLD}✓ Logged in successfully.{END}')
    driver.get('http://127.0.0.1:8000/administrator/subscriptions/')
    create_button = wait_for_element(driver, By.CLASS_NAME, 'btn-primary')
    create_button.click()
    time.sleep(2)
    print(f'\n{BLUE}{BOLD}Testing Form Validation:{END}')
    plan_name = wait_for_element(driver, By.ID, 'planName')
    price = wait_for_element(driver, By.ID, 'price')
    duration = wait_for_element(driver, By.ID, 'duration')
    create_plan_button = wait_for_element_visible(driver, By.CLASS_NAME, 'swal2-confirm')
    plan_name.clear()
    plan_name.send_keys('Basic')
    price.clear()
    price.send_keys('100')
    duration.clear()
    duration.send_keys('30')
    client_features = driver.find_elements(By.CLASS_NAME, 'client-feature')
    freelancer_features = driver.find_elements(By.CLASS_NAME, 'freelancer-feature')
    client_features[0].click()
    freelancer_features[0].click()
    time.sleep(2)
    if create_plan_button.get_attribute('disabled'):
        print(f'{GREEN}{BOLD}✓ Test Passed: Create Plan button is enabled with valid
data.{END}')
    else:
        print(f'{RED}{BOLD}X Test Failed: Create Plan button is disabled despite valid
data.{END}')
```

except Exception as e:

```
print(f"{RED}{BOLD}X Test Failed: {str(e)}{END}")
```

finally:

```
driver.quit()
```

Screenshot

Figure 30: subscription plan creation form with invalid data

```
PS C:\Users\LENOVO\OneDrive\Desktop\freelancehub> & "C:/Program Files/p
yhton/python.exe" c:/Users/LENOVO/OneDrive/Desktop/freelancehub/tests/t
est_subscription_plan.py

DevTools listening on ws://127.0.0.1:54659/devtools/browser/4e70f923-0f
ab-4cc9-9b37-a5b76a9eaa68
✓ Logged in successfully.

Testing Form Validation:
✓ Test Passed: Create Plan button is enabled with valid data.
```

Figure 31: subscription plan creation form Test result

Test Report

Test Case 6	
Project Name: FreelanceHub	
New Subscription Plan Creation Test Case	
Test Case ID: Test_6	Test Designed By: Varsha Shaji
Test Priority(Low/Medium/High):High	Test Designed Date: 09/093/2025

Module Name: New Subscription plan Creation Page			Test Executed By: Ms. Meera Rose Mathew		
Test Title: Verify Creating new subscription plan with invalid credentials			Test Execution Date: 10/03/2025		
Description: New Subscription Plan Creation form Testing					
Pre-Condition: Providing Invalid credentials					
Step	Test Step	Test Data	Expected Result	Actual Result	Status(Pass/Fail)
1	Navigate to login page		Display login page	Login page displayed	Pass
2	Valid details are entered	email: admin@gmail.com password: Admin@12	Admin should be able to login	Admin navigate to his dashboard	Pass
3	Click on login Button				
4	Navigate to subscription page and click on create new plan button	Display subscription page and click on create new plan button page	Display Form for plan creation	Form for plan creation displayed	Pass
5	Data entered	Plan name: Basic Price:100	User shouldn't be able to create plan	Shows error in form and wouldn't be able to create plan.	Pass
6	Click on submit Button				
Post-Condition: Freelancer cannot create plan because of invalid credentials					

CHAPTER 6

IMPLEMENTATION

6.1 INTRODUCTION

Software design becomes a functional product by developing it through programming and testing operations within the same phase. During this phase developers both verify requirements fulfillment and apply strategies to maintain the software along with making it scalable. The target platform receives the product after stakeholders cooperate for proper planning. A system that meets these requirements becomes a stable and functional product which remains easy to maintain at its current state.

6.2 IMPLEMENTATION PROCEDURES

Software implementation begins with planning, goal setting, task assignment, and setup. Key steps include:

- **Development & Testing:** Developers build and test the software for functionality.
- **User Acceptance Testing:** Real users confirm it meets their needs.
- **Deployment:** Final version is released to users.
- **Post-Deployment Testing:** System is tested in live conditions.
- **Support & Maintenance:** Issues are fixed and improvements are made.
- **Feedback & Improvement:** User input guides future enhancements.
- **Collaboration:** Ongoing communication ensures smooth execution.

6.2.1 User Training

Effective software implementation depends heavily on user training because it allows end-users to handle the system effectively. There are four standard procedures for user training:

- **Training Needs Assessment:** Identify which software features users need to learn.
- **Training Plan Development:** Create a plan with goals, methods, and materials.
- **Training Delivery:** Conduct training via classes, online modules, or one-on-one sessions.
- **Training Evaluation:** Gather user feedback to measure effectiveness.
- **Follow-Up Training:** Offer additional sessions to address remaining skill gaps.

User training programs empower end-users to operate new software effectively thus raising productivity while enabling smooth business transitions. The implementation process requires enough training and support to reach successful adoption and generates better business performance.

6.2.2 Training on the Application Software

The effectiveness and efficiency of end-users depend on receiving training for application software

applications. The key steps include:

- The first step is orientation through which staff members receive a basic overview about software capabilities together with its advantages to spark their enthusiasm.
- Through practical training opportunities users gain experience by conducting real sessions or testing procedures within the software framework.
- Training sessions adapt to user job roles in order to keep content relevant to their work responsibilities.
- The training program includes user-specific materials that provide easy-to-follow user-friendly information.
- Monitoring training success and improving effectiveness occurs during the evaluation step.

User confidence increases together with workplace productivity while the software implementation transition becomes easier through effective training methods.

6.2.3 System Maintenance

The system maintenance function ensures both operational efficiency and business requirements fulfillment within a project. Key activities include:

- **Regular Updates** – Apply bug fixes, security patches, and new features to maintain efficiency.
- **Backup and Recovery** – Perform regular data backups and have a disaster recovery plan.
- **Performance Monitoring** – Track system performance to detect and resolve issues.
- **User Support** – Provide help desk assistance for user queries and technical issues.
- **Security Measures** – Implement firewalls, antivirus, and intrusion detection for protection.
- **Training and Documentation** – Offer ongoing training and update user manuals as needed.

The correct maintenance practices lead to better operational efficiency together with enhanced protection and uninterrupted business operations.

6.2.4 Hosting

Koyeb provides reliable and scalable access to the Django-based FreelanceHub project. The combination of MariaDB database service through Files.io supports automatic scaling and deployment features alongside secure storage functions. The system operates with strong protection and excellent performance through SSL encryption and database backup processes along with monitoring capabilities.

Procedure for hosting on Koyeb.app:

- Create an account at Koyeb and log in.

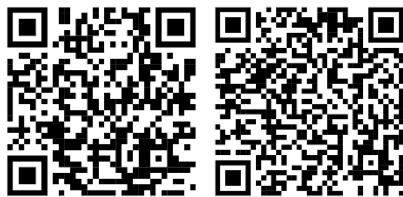
- Click "Create Service" and connect your GitHub repository.
- Ensure Django runs locally and create requirements.txt using `pip freeze > requirements.txt`.
- Add a Procfile with `web: gunicorn myproject.wsgi --bind 0.0.0.0:$PORT` (replace myproject).
- Set `ALLOWED_HOSTS` in settings.py and configure the database credentials for MariaDB.
- Configure static files using WhiteNoise or external storage.
- Click Deploy on Koyeb and monitor logs for issues.
- Access the live app via the Koyeb-provided URL.
- Push updates to GitHub for automatic redeployment.

Hosted Website: Koyeb.app, render.com

Hosted Link:

- <https://freelancehub.koyeb.app/>
- <https://freelancehub-5tya.onrender.com/>

Hosted Link QR Code



Screenshot

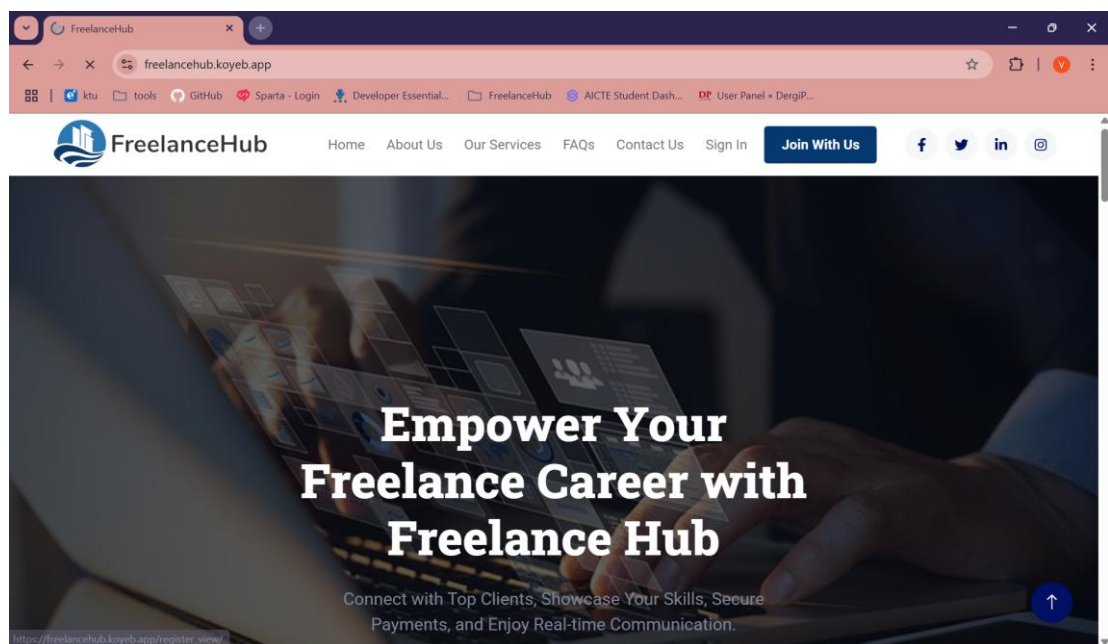


Figure 32: Hosted website

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

FreelanceHub serves as a complex Freelance Job Management System which simplifies team operations to boost communication between clients and freelancers. Both project posting services and proposal management and agile execution with Scrum framework run on the system platform. The system enables real-time chatting as it supports AI device assistance during skill assessment procedures to create individual training programs. The platform allows freelancers to create credentials that lead to joining collaborative team projects and clients acquire capabilities to plan events and quiz activities while managing employee recruitment with certification capability. Transaction security comes from the payment feedback system together with installment payments that ensure financial protection for transactions. FreelanceHub provides strong administrative options with improved premium features that establish a protected and organized interactive platform for freelance work.

7.2 FUTURE SCOPE

This includes AI-based job matching that it says could allow freelancers to discover relevant opportunities faster. Contract generation automation will form contracts faster than ever between clients and freelancers. Improved team collaboration tools will allow multi-user projects management without a hitch. Interactive learning programs, skill badges, and premium job visibility will also be available on the platform. By harnessing advanced analytics, intelligent hiring tools, and a developing AI ecosystem, FreelanceHub will deem to take a thriving, effective, and creative freelance industry to new heights.

CHAPTER 8

BIBLIOGRAPHY

REFERENCES

- Pressman, Roger S. Software Engineering: A Practitioner's Approach. McGraw-Hill Education, 2019.
- Vincent, William S. Django for Beginners: Build Websites with Python and Django. WelcomeToCode, 2018.
- Duckett, Jon. HTML and CSS: Design and Build Websites. Wiley, 2011.
- Duckett, Jon. JavaScript and jQuery: Interactive Front-End Web Development. Wiley, 2014.
- Sutherland, Jeff. Scrum: The Art of Doing Twice the Work in Half the Time. Crown Business, 2014.
- Guillebeau, Chris. The \$100 Startup: Reinvent the Way You Make a Living, Do What You Love, and Create a New Future. Crown Business, 2012.
- Beaird, Jason. The Principles of Beautiful Web Design. SitePoint, 2010.
- Greenfeld, Daniel Roy, and Greenfeld, Audrey Roy. Two Scoops of Django: Best Practices for Django 3.x. Roy Greenfeld, 2020.
- Ries, Eric. The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses. Crown Business, 2011.
- Date, C.J.. Database Design and Relational Theory: Normal Forms and All That Jazz. O'Reilly Media, 2019.

WEBSITES

- <https://www.upwork.com/>
- <https://www.freelancer.in/>
- <https://www.toptal.com/freelance-jobs>
- <https://www.guru.com/>
- <https://www.tutorialspoint.com/django/index.htm>
- <https://www.w3schools.com/django/>
- <https://www.servicescape.com/>
- <https://www.w3schools.com/django/>

CHAPTER 9

APPENDIX

9.1 Sample Code

Subscription plan management-Admin

```
{% extends 'Admin/base.html' %}

{% block 'admin_content' %}
<div class="container-fluid" style="width: 930px;">
  <div class="d-sm-flex align-items-center justify-content-between mb-4">
    <h1 class="h3 mb-0 text-gray-800">Subscription Plans</h1>
    <button class="btn btn-primary" onclick="showPlanForm()">
      <i class="fas fa-plus"></i> Create New Plan
    </button>
  </div>
  <div class="card shadow mb-4">

    <div class="card-body">
      <div class="table-responsive">
        <table class="table table-bordered" id="plansTable" width="100%" cellpadding="0">
          <thead>
            <tr>
              <th>Name</th>
              <th>Price</th>
              <th>Duration</th>
              <th>Status</th>
              <th>Actions</th>
            </tr>
          </thead>
          <tbody>
            {% for plan in plans %}
            <tr>
              <td>{{ plan.name }}</td>
              <td>₹ {{ plan.price }}</td>
              <td>{{ plan.duration_days }} days</td>
              <td>
                <div class="custom-control custom-switch">
                  <input type="checkbox" class="custom-control-input"
                    id="status_{{ plan.id }}"
                    {% if plan.is_active %}checked{% endif %}
                    onclick="return false;">
                  <label class="custom-control-label" for="status_{{ plan.id }}"
                    onclick="confirmToggleStatus('{{ plan.id }}',
document.getElementById('status_{{ plan.id }}'))">
                    {% if plan.is_active %}Active{% else %}Inactive{% endif %}
                  </label>
                </div>
              </td>
              <td>
                <button class="btn btn-sm btn-primary" onclick="showPlanForm({{ plan.id
```

```

    }})">
        <i class="fas fa-edit"></i>
    </button>
    <button class="btn btn-sm btn-info"
        onclick="showFeatures(
            '{{ plan.name }}',
            {{ plan.price }},
            {{ plan.duration_days }},
            {{ plan.is_active|lower }},
            {{ plan.can_hire_freelancer|lower }},
            {{ plan.can_create_events|lower }},
            {{ plan.project_creation_limit }},
            {{ plan.ai_freelancer_recommendations|lower }},
            {{ plan.can_participate_events|lower }},
            {{ plan.can_create_portfolio|lower }},
            {{ plan.max_proposals_limit }},
            {{ plan.ai_skill_gap_analysis|lower }},
            {{ plan.ai_project_recommendations|lower }},
            {{ plan.team_creation|lower }}
        )"><i class="fa fa-eye" aria-hidden="true"></i>

    </button>
</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
</div>
</div>
</div>

{% endblock %}

```

View.py

```

@login_required
def subscription_list(request):
    plans = SubscriptionPlan.objects.all()

    context = {
        'plans': plans
    }
    return render(request, 'Admin/subscriptions_list.html', context)

@login_required
@require_POST
def toggle_plan_status(request, pk):
    try:
        plan = SubscriptionPlan.objects.get(pk=pk)

```

```

        plan.is_active = not plan.is_active
        plan.save()
        return JsonResponse({'success': True})
    except SubscriptionPlan.DoesNotExist:
        return JsonResponse({'success': False}, status=404)

@login_required
def create_subscription_plan(request):
    if request.method == 'POST':
        try:
            plan = SubscriptionPlan.objects.create(
                name=request.POST['name'],
                price=request.POST['price'],
                duration_days=request.POST['duration_days'],
                is_active=True,

                can_hire_freelancer=request.POST.get('can_hire_freelancer') == 'true',
                can_create_events=request.POST.get('can_create_events') == 'true',
                project_creation_limit=request.POST.get('project_creation_limit', 0),
                ai_freelancer_recommendations=request.POST.get('ai_freelancer_recommendations') ==
'true',

                can_participate_events=request.POST.get('can_participate_events') == 'true',
                can_create_portfolio=request.POST.get('can_create_portfolio') == 'true',
                max_proposals_limit=request.POST.get('max_proposals_limit', 0),
                ai_skill_gap_analysis=request.POST.get('ai_skill_gap_analysis') == 'true',
                ai_project_recommendations=request.POST.get('ai_project_recommendations') == 'true',
                team_creation=request.POST.get('team_creation') == 'true',
            )
            return JsonResponse({'success': True})
        except Exception as e:
            return JsonResponse({'error': str(e)}, status=400)
    return JsonResponse({'error': 'Invalid request'}, status=400)

@login_required
def update_subscription_plan(request, pk):
    if request.method == 'POST':
        try:
            plan = SubscriptionPlan.objects.get(pk=pk)
            plan.name = request.POST.get('name')
            plan.price = request.POST.get('price')
            plan.duration_days = request.POST.get('duration_days')

            # Update features
            plan.can_hire_freelancer = request.POST.get('can_hire_freelancer') == 'true'
            plan.can_create_events = request.POST.get('can_create_events') == 'true'
            plan.project_creation_limit = request.POST.get('project_creation_limit', 0)
            plan.ai_freelancer_recommendations = request.POST.get('ai_freelancer_recommendations')
== 'true'

            plan.can_participate_events = request.POST.get('can_participate_events') == 'true'

```

```

plan.can_create_portfolio = request.POST.get('can_create_portfolio') == 'true'
plan.max_proposals_limit = request.POST.get('max_proposals_limit', 0)
plan.ai_skill_gap_analysis = request.POST.get('ai_skill_gap_analysis') == 'true'
plan.ai_project_recommendations = request.POST.get('ai_project_recommendations') ==
'true'
plan.team_creation = request.POST.get('team_creation') == 'true'

plan.save()
return JsonResponse({'success': True})
except Exception as e:
    return JsonResponse({'error': str(e)}, status=400)
return JsonResponse({'error': 'Invalid request'}, status=400)

```

Teams management

```

{% extends 'freelancer/base.html' %}
{% load static %}

{% block 'freelancer_content' %}
<div class="row">
    <div class="col-12 grid-margin">
        <div class="card">
            <div class="card-body">
                <h4 class="card-title">Teams Management</h4>

                <div class="mt-3 text-right">
                    <button class="btn btn-primary" id="createTeamBtn" data-toggle="modal" data-
target="#createTeamModal">
                        <i class="fas fa-users"></i> Create Team
                    </button>
                </div>
                <div class="table-responsive">
                    <table class="table table-hover">
                        <thead>
                            <tr>
                                <th>Team Name</th>
                                <th>Created Date</th>
                                <th>Actions</th>
                            </tr>
                        </thead>
                        <tbody>
                            {% for team in owned_teams %}
                                <tr>
                                    <td>{{ team.name }}</td>
                                    <td>{{ team.created_at|date:"M d, Y" }}</td>
                                    <td>
                                        <a href="{% url 'freelancer:manage_team' team.id %}" class="btn btn-sm
btn-outline-info">Manage Team</a>
                                        <button class="btn btn-sm btn-outline-primary" data-toggle="modal" data-
target="#editTeamModal{{ team.id }}" onclick="populateEditModal({{ team.id }}, '{{ team.name
}}');">Edit</button>

```

```

        <form method="POST" action="{% url 'freelancer:delete_team' team.id %}"
style="display:inline;">
            {% csrf_token %}
            <button type="submit" class="btn btn-sm btn-outline-danger"
onclick="return confirm('Are you sure you want to delete this team?');">Remove</button>
        </form>
    </td>
</tr>
<!-- Edit Team Modal -->
<div class="modal fade" id="editTeamModal{{ team.id }}" tabindex="-1"
role="dialog">
    <div class="modal-dialog" role="document">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Edit Team</h5>
                <button type="button" class="close" data-dismiss="modal">
                    <span>&times;</span>
                </button>
            </div>
            <form method="POST" action="{% url 'freelancer:edit_team' team.id
%}">
                {% csrf_token %}
                <div class="modal-body">
                    <div class="form-group">
                        <label>Team Name</label>
                        <input type="text" class="form-control" name="team_name"
id="editTeamName{{ team.id }}" required>
                    </div>
                </div>
                <div class="modal-footer">
                    <button type="button" class="btn btn-secondary" data-
dismiss="modal">Close</button>
                    <button type="submit" class="btn btn-primary">Update
Team</button>
                </div>
            </form>
        </div>
    </div>
</div>
<div>
    {% empty %}
<tr>
    <td colspan="3" class="text-center">No teams found</td>
</tr>
    {% endfor %}
</tbody>
</table>
</div>
</div>
</div>
</div>
</div>

```

```

<!-- Create Team Modal -->
<div class="modal fade" id="createTeamModal" tabindex="-1" role="dialog">
  <div class="modal-dialog" role="document">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Create New Team</h5>
        <button type="button" class="close" data-dismiss="modal">
          <span>&times;</span>
        </button>
      </div>
      <form method="POST" action="{% url 'freelancer:create_team' %}">
        {% csrf_token %}
        <div class="modal-body">
          <div class="form-group">
            <label>Team Name</label>
            <input type="text" class="form-control" name="team_name" id="teamNameInput"
required>
            <div id="error-message" style="color: red; display: none;">Team name already
exists!</div>
          </div>
        </div>
        <div class="modal-footer">
          <button type="button" class="btn btn-secondary" data-dismiss="modal"
id="closeButton">Close</button>
          <button type="submit" class="btn btn-primary" id="submitButton">Create
Team</button>
        </div>
      </form>
    </div>
  </div>
</div>

<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<script>
  $(document).ready(function() {
    $('#teamNameInput').on('keyup', function() {
      const teamName = $(this).val();
      if (teamName) {
        $.ajax({
          url: '{% url "freelancer:check_team_name" %}',
          data: {
            'team_name': teamName
          },
          dataType: 'json',
          success: function(data) {
            if (data.exists) {
              $('#error-message').show(); // Show error message
              $('#submitButton').prop('disabled', true); // Disable submit button
            } else {
              $('#error-message').hide(); // Hide error message
            }
          }
        });
      }
    });
  });
</script>

```



```

        $('#submitButton').prop('disabled', false); // Enable submit button
    }
}
});
} else {
    $('#error-message').hide(); // Hide error message if input is empty
    $('#submitButton').prop('disabled', false); // Enable submit button
}
});
});
</script>

{% endblock %}

```

Views.py

```

from django.contrib import messages
from django.shortcuts import redirect
from .models import Team, TeamMember
import uuid

def create_team(request):
    if request.method == 'POST':
        team_name = request.POST.get('team_name')
        try:
            while True:
                join_code = str(uuid.uuid4())[:8]
                if not Team.objects.filter(join_code=join_code).exists():
                    break

            team = Team.objects.create(
                name=team_name,
                created_by=request.user,
                join_code=join_code
            )
            TeamMember.objects.create(
                team=team,
                user=request.user,
                role='PROJECT_MANAGER'
            )
            messages.success(request, f'Team "{team_name}" created successfully! Join code: {team.join_code}')
        except Exception as e:
            messages.error(request, f'Failed to create team: {str(e)}')
        return redirect('freelancer:my_teams')

def edit_team(request, team_id):
    team = get_object_or_404(Team, id=team_id)
    if request.method == 'POST':
        team_name = request.POST.get('team_name')

```

```

if team_name:
    team.name = team_name
    team.save()
    return redirect('freelancer:my_teams')

return redirect('freelancer:my_teams')

def delete_team(request, team_id):
    team = get_object_or_404(Team, id=team_id)
    if request.method == 'POST':
        team.delete()
        return redirect('freelancer:my_teams')
    return redirect('freelancer:my_teams')

```

9.2 Screen Shots

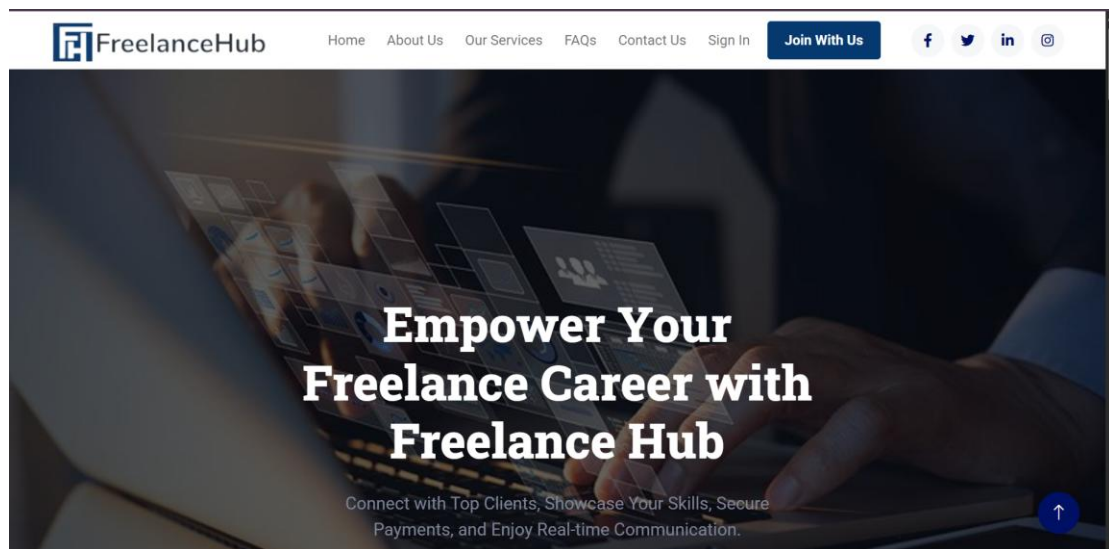


Figure 33: Landing Page

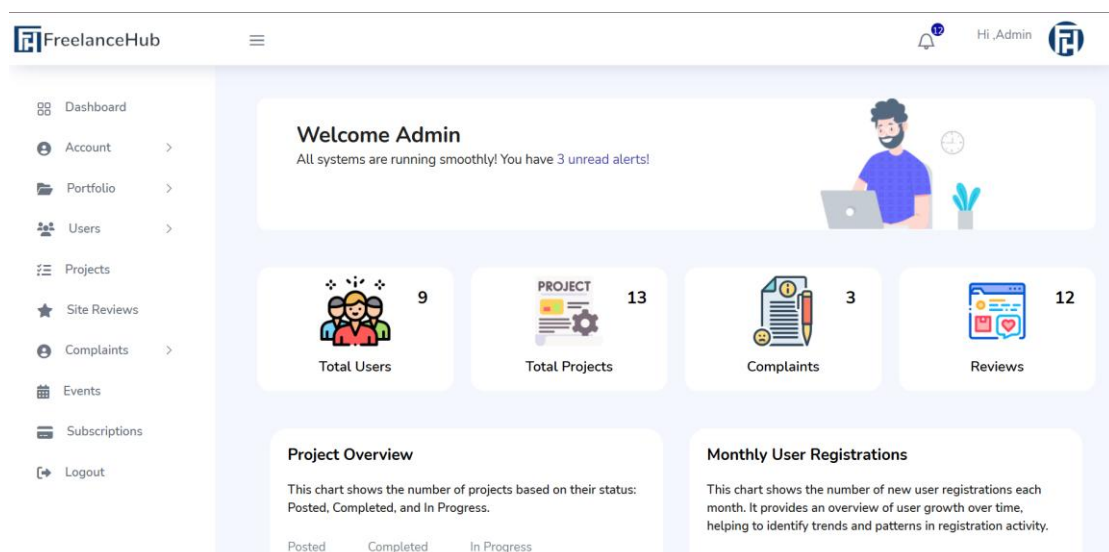


Figure 34: Admin Dashboard

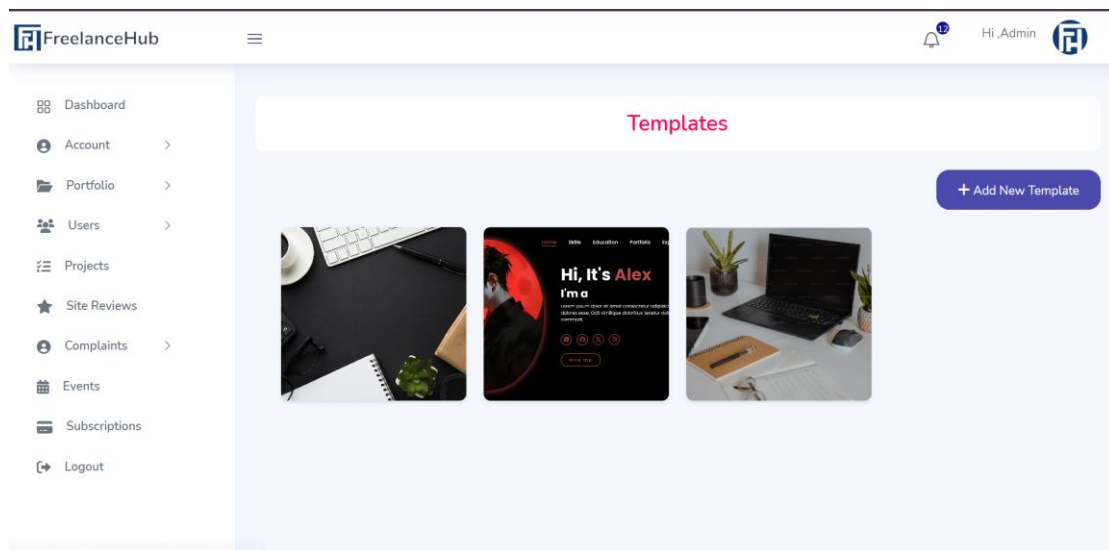


Figure 35: Portfolio Templates view (Admin Side)

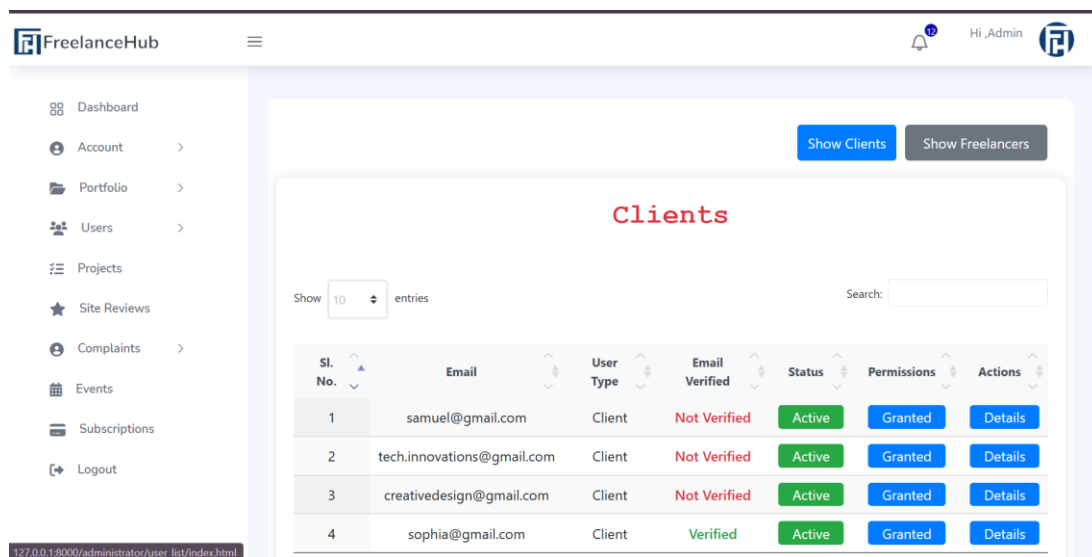


Figure 36: User Management Page(Admin Side)

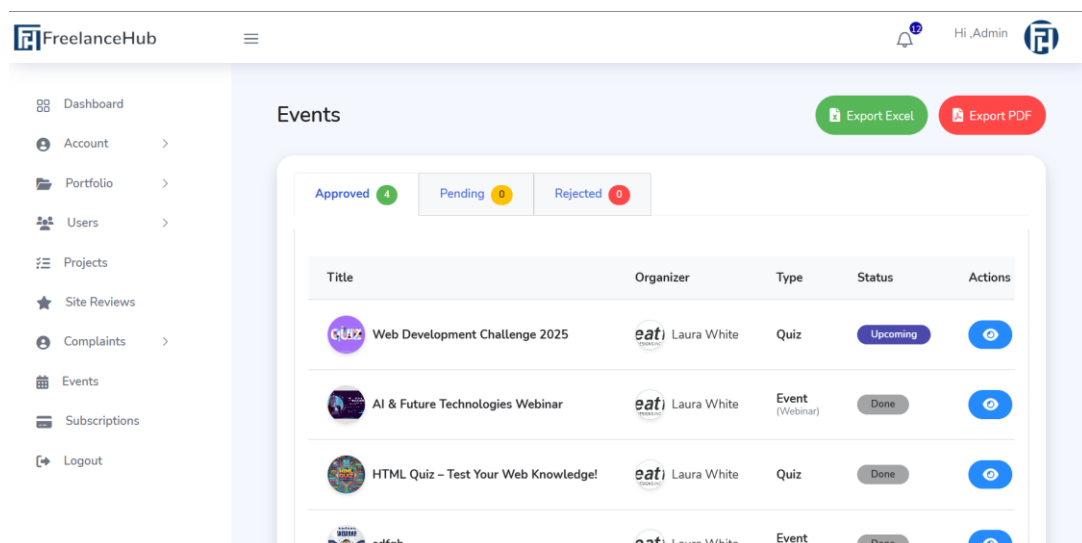


Figure 37: Event Management page

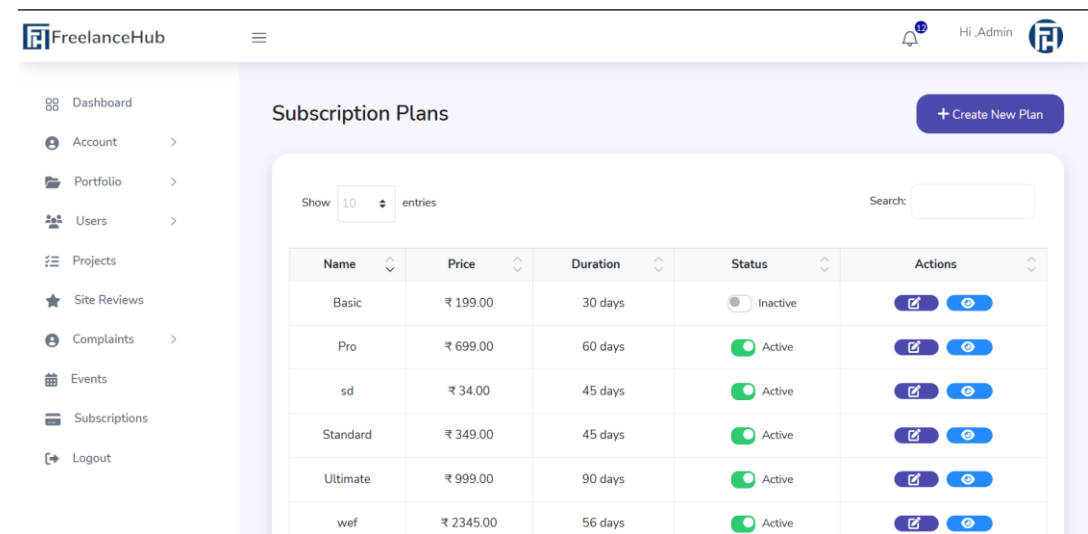


Figure 38: Subscription Plan Management page (Admin Side)

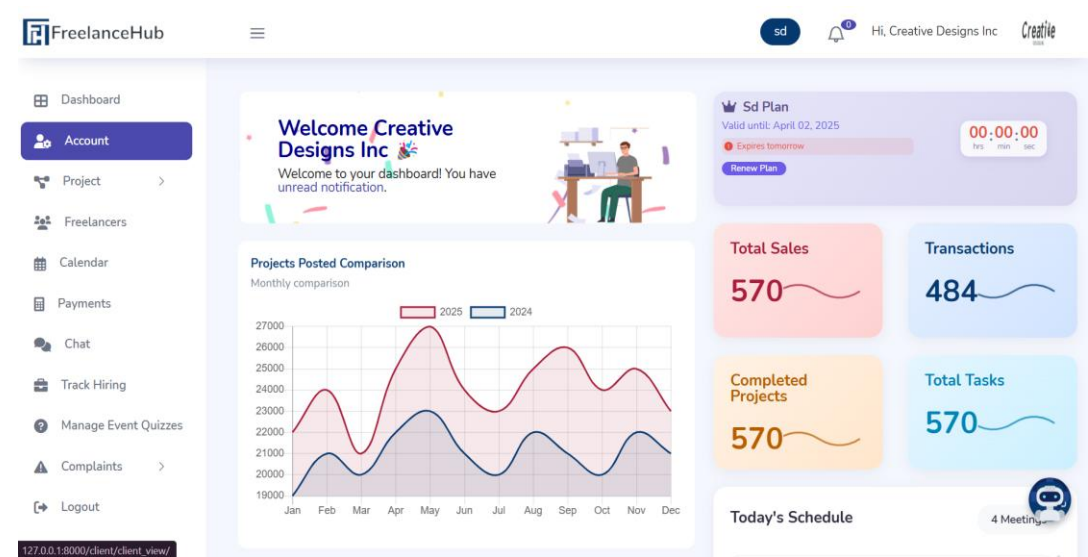


Figure 39: client dashboard

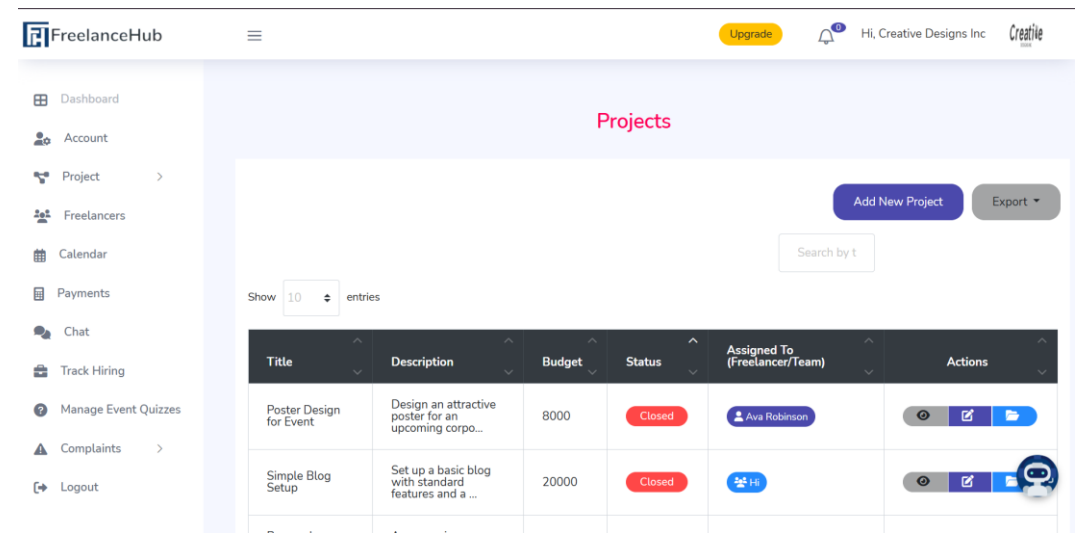


Figure 40: Project List Page(client side)

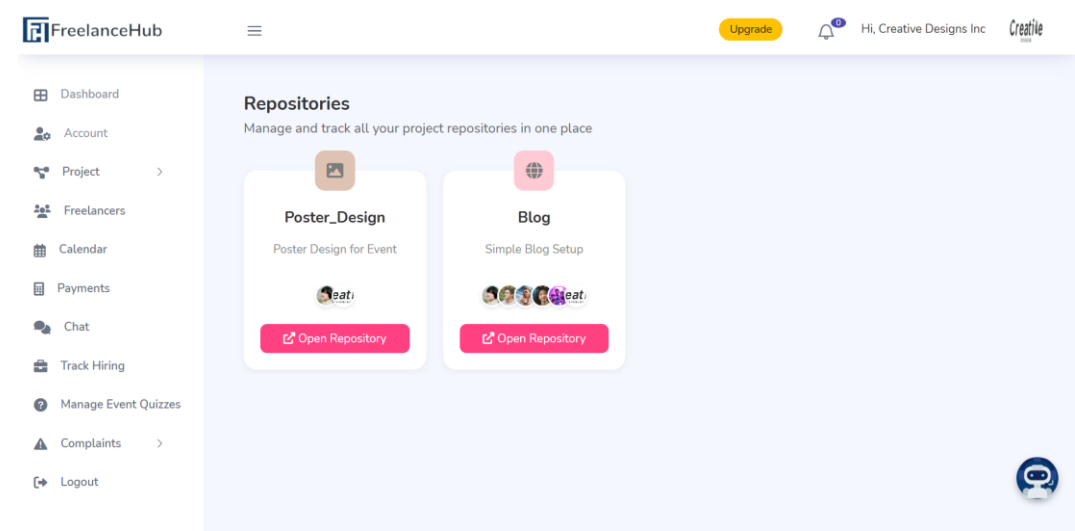


Figure 41: Project List Page(client side)

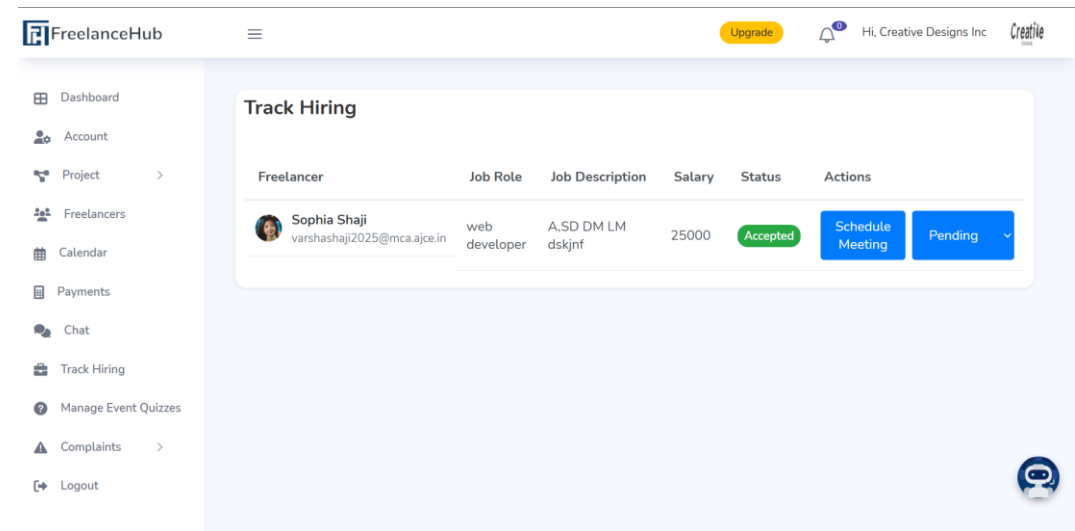


Figure 42: Hiring Tracking Page(client side)

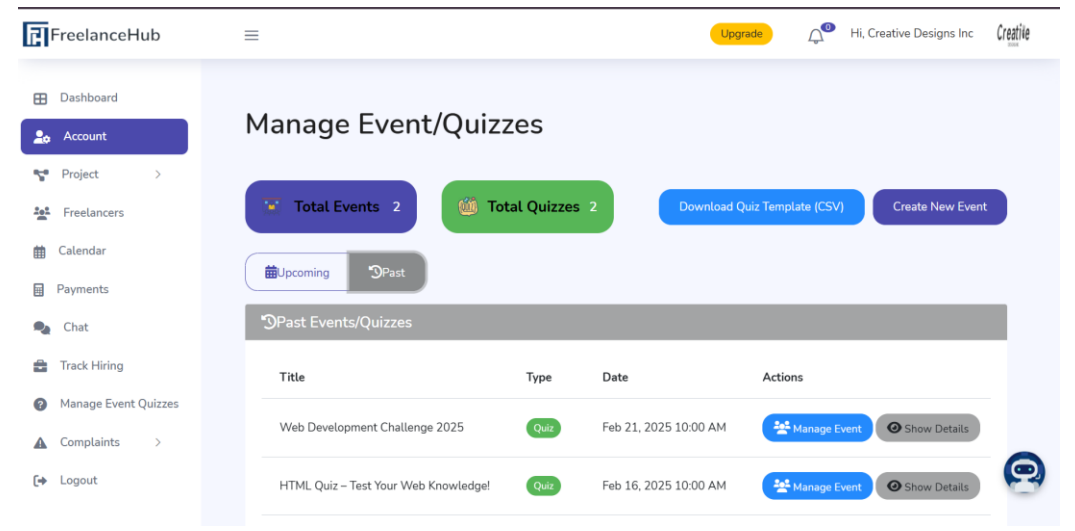


Figure 43: Event and Quiz management page(client side)

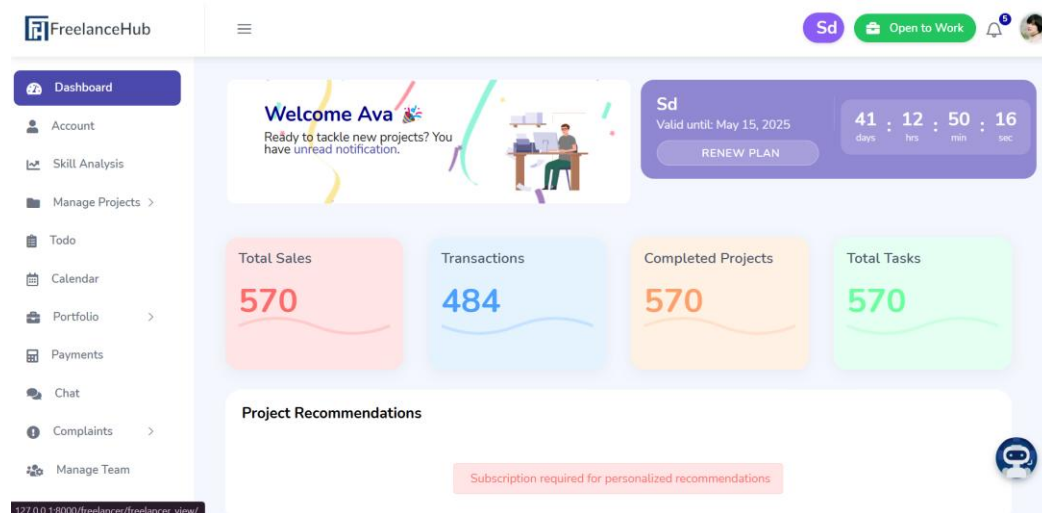


Figure 44: Freelancer Dashboard

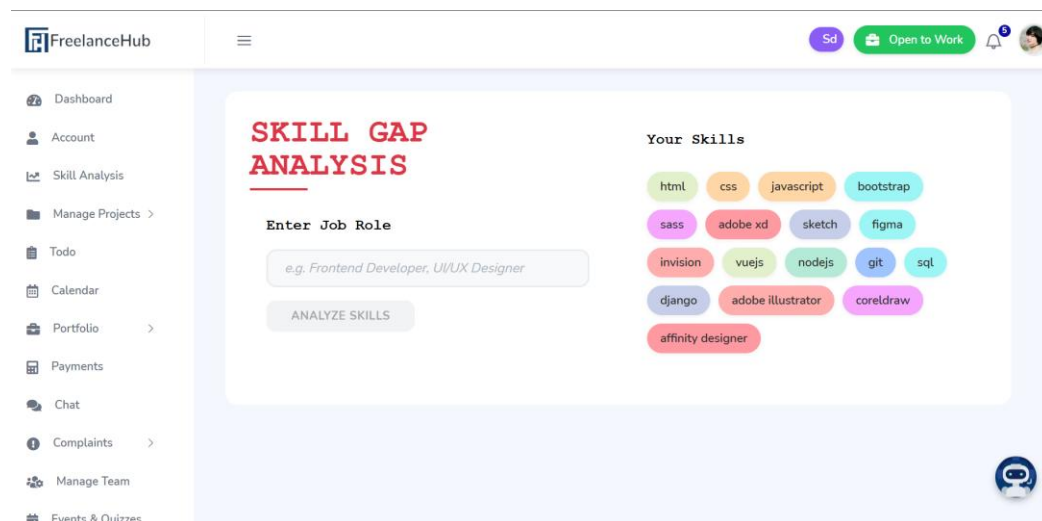


Figure 45: Skill gap analysis page

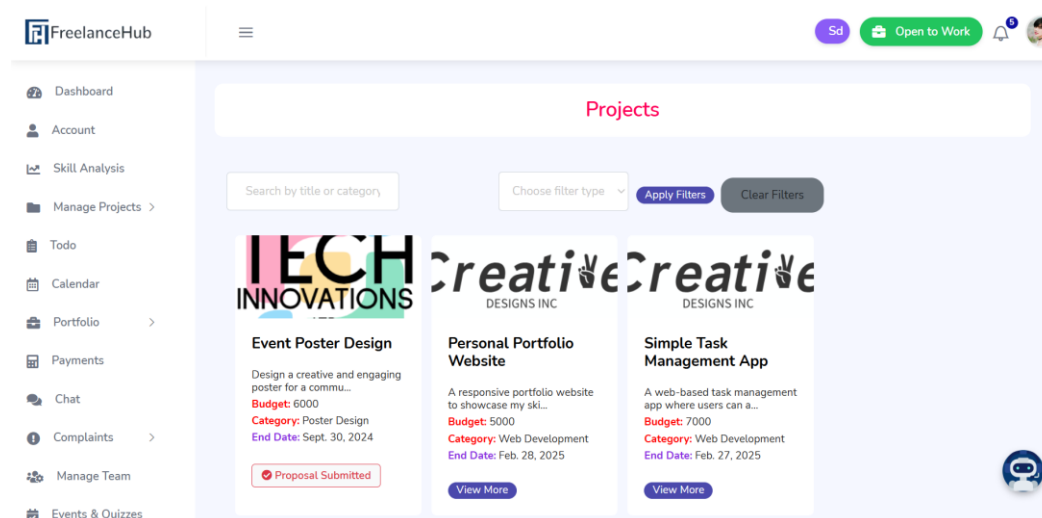


Figure 46: Project List page (freelancer side)

FreelanceHub

Hi

Complete Team - Ready for submitting Proposals for projects

Team Invitation

Member	Role	Status	Joined Date
You vshaji596@gmail.com	Project Manager & Team Leader	Active	Jan 16, 2025
Daniel Miller daniel.miller@gmail.com	Designer (UI/UX)	Active	Jan 24, 2025
Noah Wilson noah.wilson@gmail.com	Backend Developer	Active	Jan 24, 2025

Figure 47: Team management page

9.3 Git Log

varshashaji3 / FreelanceHub

Code Issues Pull requests Actions Projects Wiki Security 2 Insights

Commits

main

All users All time

Commits on Aug 11, 2024

- Delete FreelanceHub directory**
Verified de60cb0 <> ...
varshashaji3 authored on Aug 11, 2024

Commits on Aug 5, 2024

- Delete static/hi**
Verified c7bd87a <> ...
varshashaji3 authored on Aug 5, 2024
- Add files via upload**
Verified 8a9aa95 <> ...
varshashaji3 authored on Aug 5, 2024
- Create hi**
Verified 349363a <> ...
varshashaji3 authored on Aug 5, 2024
- Delete static**
Verified 35aa55e <> ...
varshashaji3 authored on Aug 5, 2024
- Create static**
Verified 8da5c75 <> ...
varshashaji3 authored on Aug 5, 2024
- Add files via upload**
Verified 00cd6cf <> ...
varshashaji3 authored on Aug 5, 2024

Add files via upload
Verified 2ff5141   ...
 varshashaji3 authored on Aug 5, 2024

Add files via upload
Verified da57e94   ...
 varshashaji3 authored on Aug 5, 2024

Add files via upload
Verified 3ab728c   ...
 varshashaji3 authored on Aug 5, 2024

Add files via upload
Verified 86ad201   ...
 varshashaji3 authored on Aug 5, 2024

Add files via upload
Verified 0d041d5   ...
 varshashaji3 authored on Aug 5, 2024

🔍 Commits on Jul 26, 2024

Add files via upload
Verified 0afb861   ...
 varshashaji3 authored on Jul 26, 2024

Add files via upload 
Verified c344139   ...
 varshashaji3 authored on Jul 26, 2024

🔍 Commits on Jun 20, 2024

2nd commit
Verified d5d373f   ...
 varshashaji3 authored on Jun 20, 2024

🔍 Commits on Jun 15, 2024

Abstract
Verified f8d1fffd   ...
 varshashaji3 authored on Jun 15, 2024

varshashaji3 / FreelanceHub

🔍

📄

+

🔄

🔗

📧

👤

<> Code

🕒 Issues

🔗 Pull requests

🔄 Actions

📁 Projects

📖 Wiki

🛡 Security 2

📈 Insights

Commits

main

All users

All time

🕒 Commits on Apr 4, 2025

project completed

18e5ddf 📄 <> ...

👤 varshashaji3 committed 11 minutes ago

🕒 Commits on Mar 17, 2025

adding new files

587c186 📄 <> ...

👤 varshashaji3 committed 3 weeks ago

🕒 Commits on Feb 20, 2025

upto review

197dcbd 📄 <> ...

👤 varshashaji3 committed on Feb 20

🕒 Commits on Feb 13, 2025

chabot,client hiring freelancer,client events and quizzes are done

cbb22c6 📄 <> ...

👤 varshashaji3 committed on Feb 13

gerge branch 'main' of <https://github.com/varshashaji3/FreelanceHub> 📄

fd5df12 📄 <> ...

👤 varshashaji3 committed on Feb 13

chabot,client hiring freelancer,client events and quizzes are done

b57e8Fa 📄 <> ...

👤 varshashaji3 committed on Feb 13

Commits on Jan 29, 2025

docs uploaded	7bfdda1			...
 varshashaji3 committed on Jan 29				
team created,group chat,upto contract accepted done	6ce9c18			...
 varshashaji3 committed on Jan 29				

Commits on Nov 13, 2024

Add files via upload	Verified	e2b2d16			...
 varshashaji3 authored on Nov 13, 2024					
Add files via upload	Verified	dfc78a8			...
 varshashaji3 authored on Nov 13, 2024					

Commits on Nov 5, 2024

Project Done	Verified	0832e97			...
 varshashaji3 authored on Nov 5, 2024					
project done	Verified	b284a95			...
 varshashaji3 authored on Nov 5, 2024					
project done	Verified	819d348			...
 varshashaji3 authored on Nov 5, 2024					
project done	Verified	a7e3e88			...
 varshashaji3 authored on Nov 5, 2024					
project done	Verified	52b6127			...
 varshashaji3 authored on Nov 5, 2024					

Commits on Oct 8, 2024

invoice generation done	824281a			...
 varshashaji3 committed on Oct 8, 2024 ·  0 / 1				

Commits on Oct 4, 2024

new selenium tests,cover_image for portfolio done	50b7222			...
 varshashaji3 committed on Oct 4, 2024 ·  0 / 1				

Commits on Oct 3, 2024

refunding done	3111380			...
 varshashaji3 committed on Oct 3, 2024				

Commits on Sep 29, 2024

modifications done,testing done

d4d1d65  

...

 varshashaji3 committed on Sep 29, 2024

Commits on Sep 23, 2024

NLP implementation done

7c77d5c  

...

 varshashaji3 committed on Sep 23, 2024

Commits on Sep 17, 2024

chat ,complaints done

7437dc3  

...

 varshashaji3 committed on Sep 17, 2024

Commits on Sep 6, 2024

chat ,complaints done

3d1e7d6  

...

 varshashaji3 committed on Sep 6, 2024

Commits on Sep 2, 2024

userreview and sitereview done

5f55564  

...

 varshashaji3 committed on Sep 2, 2024

Commits on Aug 26, 2024

upto payment

5bc9aa2  

...

 varshashaji3 committed on Aug 26, 2024

Commits on Aug 13, 2024

new commit

f6fdb7b  

...

 varshashaji3 committed on Aug 13, 2024

Commits on Aug 11, 2024

calendar and notifications done

9e1c774  

...

 varshashaji3 committed on Aug 11, 2024

upto proposal approval

2f28c42  

...

 varshashaji3 committed on Aug 11, 2024

Delete manage.py

 ce79628  

...

 varshashaji3 authored on Aug 11, 2024

Delete templates directory Verified ebc9f5c ... varshashaji3 authored on Aug 11, 2024
Delete static directory Verified 6ecbb96 ... varshashaji3 authored on Aug 11, 2024
Delete media directory
Verified 9be4176 ... varshashaji3 authored on Aug 11, 2024
Delete freelancer directory Verified 9bc480f ... varshashaji3 authored on Aug 11, 2024
Delete core directory Verified e7fc562 ... varshashaji3 authored on Aug 11, 2024
Delete client directory Verified 272bde5 ... varshashaji3 authored on Aug 11, 2024
Delete administrator directory Verified 728bfa7 ... varshashaji3 authored on Aug 11, 2024

9.4 Certificates



Elite
NPTEL Online Certification
 (Funded by the MoE, Govt. of India)



This certificate is awarded to
VARSHA SHAJI
 for successfully completing the course
Cloud Computing

with a consolidated score of **66** %

Online Assignments	25/25	Proctored Exam	40.63/75
--------------------	-------	----------------	----------

Total number of candidates certified in this course: **16686**

Jul-Oct 2023
 (12 week course)


Prof. Haimanti Banerji
 Coordinator, NPTEL
 IIT Kharagpur


 Indian Institute of Technology Kharagpur

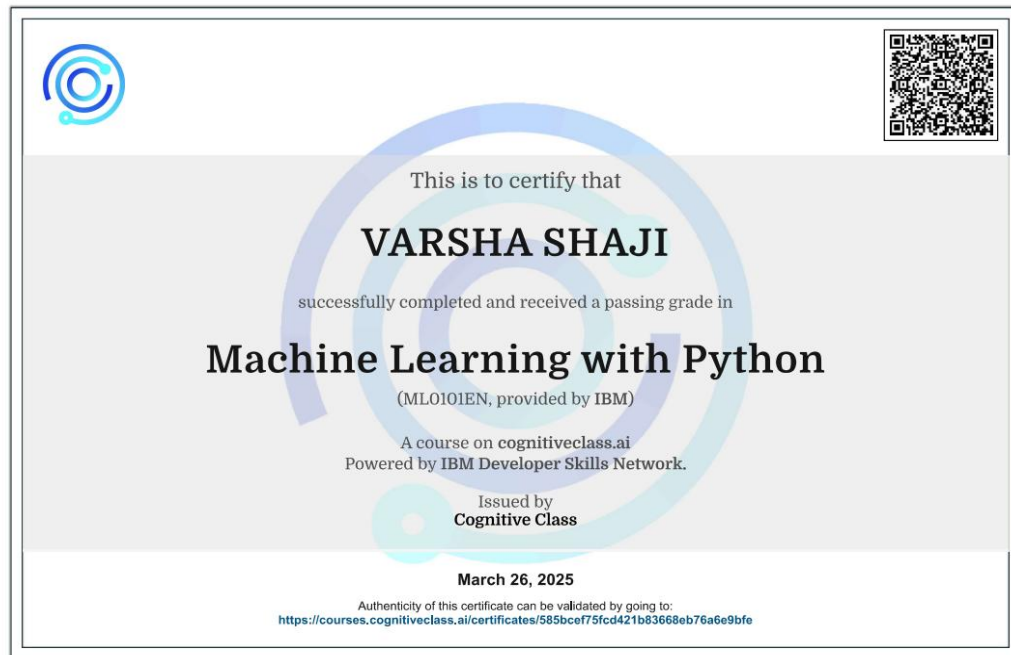


Roll No: NPTEL23CS89S540100426
 To verify the certificate 
 No. of credits recommended: 3 or 4

Figure 48: NPTEL certificate

3/26/25, 9:50 PM

IBM ML0101EN Certificate | Cognitive Class

<https://courses.cognitiveclass.ai/certificates/585bcef75fd421b83668eb76a6e9bfe>

1/1

Figure 50: ML with python certificate



Figure 51: webscraping certificate



Figure 52: AI internship certificate



Figure 53: Agile scrum master certificate

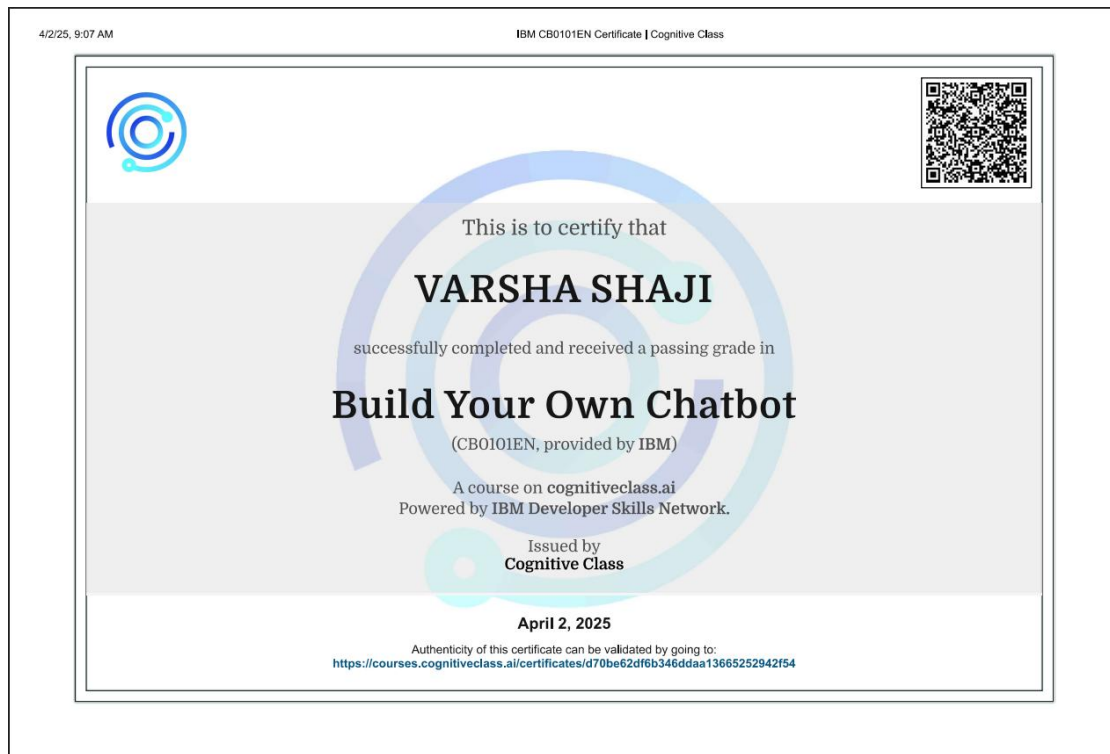


Figure 54: Build your own chatbot certificate